

MECH 421/423 Final Project Report

Bipedal Balancing Companion Robot

Prototype: Grass

Dominic Nguyen & Nicholas Yap



Abstract

Grass is a bipedal (two legs), two-wheeled self-balancing robot with wireless user control that integrates mechanical, electrical, and software design into a single dynamic platform. The system functions as an inverted pendulum on a cart, maintaining upright stability by actively driving two DC gearmotors mounted at the base of its legs. Because the system is inherently unstable, continuous feedback control is required to prevent tipping and to enable controlled motion.

The mechanical structure consists of a 3D-printed chassis designed to house electrical components, maintain sensor alignment, and protect critical hardware. The robot's dynamics are modeled using an inverted pendulum approximation, allowing for intuitive development and tuning. Grass's balance and motion are achieved using a dual cascaded PID control system. The inner control loop regulates the tilt angle, while the outer loop manages translational velocity by generating a tilt setpoint. This separation allows fast stabilization and minimizes motion drift to maintain smooth user-directed movement.

State estimation is performed using an inertial measurement unit and motor encoder, using signal processing techniques to reduce noise and drift in tilt and velocity measurements. The system runs on a dual-core ESP32 microcontroller, where multiprocessing is used to isolate the time-critical motor and PID control on one core, separating sensor acquisition and Bluetooth Low Energy communication on the other. This architecture reduces CPU overload and enhances Grass's control stability.

A custom C# WinForms user interface wirelessly communicates with the robot via Bluetooth Low Energy, allowing for real-time command input and PID tuning. By combining the mechanical design, embedded multiprocessing, self-balancing feedback control, and wireless communication, Grass represents a practical implementation of advanced control theory applied to a comprehensive, coupled, mechatronics system.

Table of Contents

Abstract.....	2
Table of Contents.....	3
1.0 Objectives.....	5
2.0 Concept and Rationale.....	6
2.1 Interest and End-User Value.....	6
2.2 Closest Alternative.....	6
2.3 Metric of Success.....	7
2.4 Polished Aspect.....	7
2.5 Unpolished Aspects.....	8
2.6 Most Critical Module.....	8
2.7 Overall Data Infrastructure.....	9
3.0 Summary of Functional Requirements.....	11
4.0 FR #1: Mechanical Design and Construction.....	13
4.1 Approach and Design.....	13
4.1.1 Chassis Design.....	14
4.1.2 Leg Design.....	16
4.2 Inputs and Outputs.....	17
4.3 Parameters.....	17
4.4 Testing and Results.....	18
4.4.1 Wheel Changes.....	18
4.4.2 Crosstalk Mitigation.....	19
4.4.3 Additional Fix - Chassis Battery Mount.....	19
5.0 FR #2: Wireless User Control.....	20
5.1 Approach and Design.....	20
5.2 Inputs and Outputs.....	22
5.3 Parameters.....	23
5.4 Testing and Results.....	24
6.0 FR #3: PID Motor Control Algorithm.....	25
6.1 Approach and Design.....	25
6.2 Inputs and Outputs.....	25
6.3 Parameters.....	26
6.4 Testing and Results.....	26
7.0 FR #4: Tilt Estimation and Data Processing Firmware.....	28
7.1 Approach and Design.....	28
7.2 Inputs and Outputs.....	29
7.3 Parameters.....	29
7.4 Testing and Results.....	30
8.0 FR #5: Circuit Design.....	31

8.1 Approach and Design.....	31
8.2 Inputs and Outputs.....	32
8.3 Parameters.....	32
8.4 Testing and Results.....	33
9.0 System Evaluation.....	34
9.1 Self-Balancing Test.....	34
9.2 Movement Test.....	34
9.3 Bluetooth Proximity Test.....	34
9.4 Impulse Recovery Test.....	35
9.5 Turning Test [Reach Goal].....	35
10.0 Reflections.....	36
10.1 Dominic.....	36
10.2 Nicholas.....	38
APPENDIX A - C# WinForms UI Code.....	41
APPENDIX B - ESP32 Firmware.....	56
APPENDIX C - Key Resources.....	68

1.0 Objectives

The overall goal of this project was to design and build a remote-control self-balancing robot that could serve as the foundation for a compact companion robot capable of assisting with carrying bags and personal accessories. Our long-term vision was to develop a commercial-level, dynamically stable bipedal robot with actuated hip and knee joints. It would be capable of performing complex maneuvers and traversing obstacles like stairs. A system of this level can be summarized as an inverted double-pendulum with a multi-PID stabilization control system.

Given the project's timeline and complexity constraints, we refined the scope to focus on a rigid-leg inverted pendulum robot, where its balance is achieved solely through actuating its wheels. The planned system included a custom 3D-printed mechanical chassis, dual DC motors, an embedded control system running on an ESP-WROOM-32 microcontroller, PID control for self-balancing, onboard state estimation using an IMU and wheel encoders, and a wireless C# graphical user interface for real-time user control.

We successfully designed, fabricated, and integrated the full mechanical, electrical, and software system. The robot is capable of autonomous self-balancing, forward and backwards motion, directional control, and wireless communication with a desktop interface. Optimizing the original plan further, smoother closed-loop control was achieved using cascaded PID controllers with multiprocessing to separate time-critical motor control from sensing and communication tasks.

However, several aspects of the project were not fully polished. Smooth, precise turning remains limited due to motor backlash and motor control limitations imposed by the implementation of the outer velocity control loop PID. Additionally, a small steady-state drift persists, indicating the need for further controller refinement and upgrade requirements for a BLDC to reduce backlash and improve symmetric motor response. Despite these setbacks and additional work required to achieve a fully polished system, the robot more than encapsulates the vision of the original concept.

2.0 Concept and Rationale

2.1 Interest and End-User Value

"What is conceptually interesting about your project? Why is it worth your time to do it? What is the value of your product to the end-user?"

The primary concept of interest is the mechanism to self-balance. It requires a complex control system utilizing PIDs with immediate reward (balancing) and consequence (falling). It tests our understanding of mechatronics systems and ability to tune parameters, as we receive real-time responses to the changes implemented onto Grass.

But why is it worthwhile? Apart from being a system that validates our academic learnings and troubleshooting skills, it is a step forward towards the idea of "Mechatronics." It is not often where we may work on a complete product, from the firmware, software, hardware, and mechanical design. However, Grass encompasses all of that and allows us to dive into the world of control theory and robotic control.

The value of Grass is that it is a step forward (for us, the designers) towards creating a compact companion robot that can assist with load-carrying on a variety of terrains and slopes. Typical rover designs can lift heavier objects, but diving into self-balancing and auto-leveling lets us solve situations where we need to keep loads upright (e.g. drinks).

2.2 Closest Alternative

"What is the closest alternative to your product?"

The closest commercially-available alternative to our product is a robot named TRON1 by [LimX Dynamics](#). TRON1 is a multi-bodal bipedal robot that can follow a user, has modular foot-ends for different terrains, and has advanced movement across a variety of terrains.

Similarities:

Our product follows the same principles and overall structure as TRON1. Both are bipedal robots and both require software control of motors to stay balanced while moving.

Differences:

The TRON1 has a much larger range of capabilities than our product, Grass, has. With modular mounting for different kinds of legs or expansions for robotic arms, sensors, etc. TRON1 has much more to offer. **However**, what Grass provides is simplicity, cheaper costs, and a smaller-scale. TRON1 utilizes reinforcement learning to complete complex tasks with a standing height of around half the average adult human. Whereas, Grass is less than half the size and provides just as much companionship.

2.3 Metric of Success

"What is the metric of success for your product?"

The main metric of success is the ability of the robot to balance on both legs for an extended period of time (over 5 minutes). The method of measurement was timing the duration the robot could stay standing and balanced after turning on. Through the Mechatronics Final Demo, Grass stood for over 5 minutes and validated the success of the project.

Additional system-level tests were conducted to ensure safe operation of the robot, its ability to move, and disturbance recovery. This involved testing the aspects of our most polished concept (designing for safety) by letting Grass fall, hitting the bumpers and watching Grass recovery, and checking the emergency off/on switch.

2.4 Polished Aspect

"What aspect of your project did you choose to develop into a polished finished product? Why? Were you successful?"

As an important aspect of the project and in overall robotics operation, we chose to polish Grass's components to be designed for safety. As an interactable robot within close-proximity to humans, any falls, collisions, or mismanaged electronics can be dangerous. Therefore, extra effort and testing was put towards designing a control system and firmware with robust safety measures (e.g. staying balanced while disconnected from PC), wire management, battery protection, emergency shut offs, and collision protection.

This was successful as we managed to incorporate a variety of safety measures into Grass's design that promote accessibility and protection, such as:

- TPU-Printed Bumpers for the front and back of the chassis to handle collisions and falls.

- TPU-Printed bumpers within the chassis that press on the battery to provide shock protection and ensure secure battery attachment during operation.
- Emergency On/Off switch directly connected from the battery to the rest of the electronics that is accessible on the side of the chassis for convenient disconnect from the LiPo battery.
- Continuous operation and embedded (onboard) self-balancing control system within Grass so that user-control is not required for Grass to stay upright and prevent damaging falls.
- Wire slots in legs and enclosed chassis to manage wires and keep high voltage wires away from contact, preventing accidental damage to the robot or surroundings.
- Smoothed handles and holes included in the 3D-printed chassis to visually monitor electronics and quick access to the LiPo battery monitoring wire to check battery voltage.

2.5 Unpolished Aspects

"What aspects of your project did you not develop to a finished product?"

The following aspects of the project were not developed into a finished product:

- Robot movement has been kept to forward and backwards movement. Turning [our reach goal] has been achieved, but is done by slowly pivoting the robot around the left wheel due to limits with the current cascaded PID setup. Further polish for turning by varying wheel speed or wheel directions can be implemented in the future by adjusting the outer PID.
- Balance in the robot has been achieved, but there is a small drift in translation due to motor backlash and further required fine-tuning.
- The circuit design has been kept rudimentary and focuses on functionality/testing rather than a sleek design. A PCB was not created for this project due to the use of multiple chip modules rather than embedded circuitry.

2.6 Most Critical Module

"What was your most critical module and why?"

The balance of the robot is the most critical module of the project. In particular, this refers to the embedded control system on the ESP-32 and the interaction between the system's sensors for I/O. To demonstrate the most critical module, we displayed Grass's ability to mitigate error in its tilt setpoint by actuating its motors to adjust the wheels, staying upright.

Starting this project, we had little to no hands-on experience with setting up a wireless data system and PIDs in C# and firmware. These were significant hurdles in the project, and demonstrated the difficulties of self-balancing and tuning. Without balancing, the robot is unable to move or function as intended. And, if done wrong, can lead to a series of robot falls and collisions which can endanger its components and surroundings.

2.7 Overall Data Infrastructure

"Describe your overall data infrastructure."

The overall data infrastructure consists of five main components as listed below:

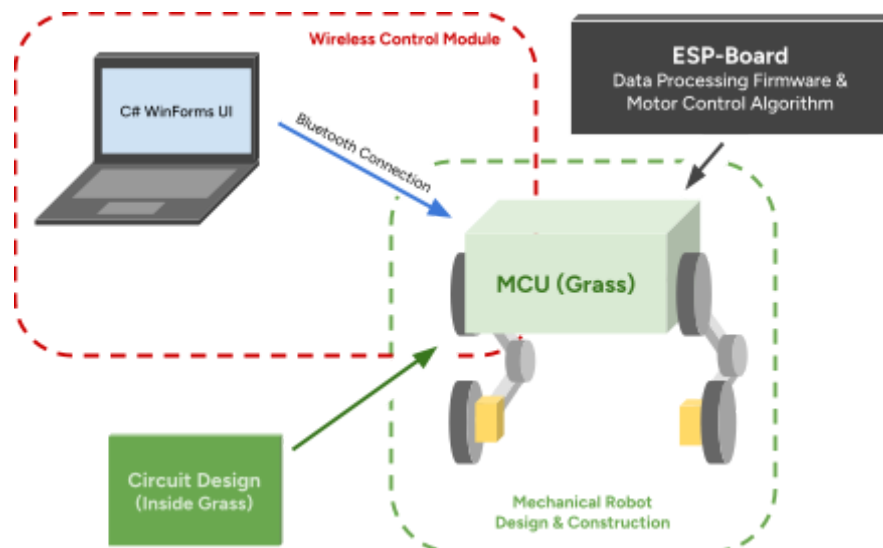
- **Wireless Data Communication**
 - Serial Bytes in the form of strings are wirelessly transmitted to the ESP-WROOM-32 microcontroller using a Bluetooth Low Energy (BLE) connection between a PC and the ESP.
 - ESP-32 has Arduino IDE Firmware to incorporate BLE communication. The ESP-32 acts as a server and broadcasts its service as "Grass" which bluetooth readers can access to take control of using a predetermined UID key.
 - The C# WinForms UI opens up the laptops/PCs bluetooth capabilities and pairs to the broadcasting ESP-32 as a BLE Client. After reading the ESP's service, the C# can access and send data across the data channel by utilizing a custom UID key.
 - The C# UI sends strings in the form of serial bytes, such as "255 10 05" to control the movement of Grass or tune its PID values using a state-machine embedded in the ESP's firmware.
- **Accelerometer & Gyroscope Data**
 - Grass's acceleration and angular velocity are transmitted from an IMU to the ESP using a SPI communication protocol to feed the tilt angle of the robot to the PID control system.
 - The inertial measurement unit (IMU) has an accelerometer and gyroscope that measures the acceleration and angular velocity of Grass. The ESP firmware uses a Madgwick filter to determine the tilt angle from these measurements.
- **Encoder Data**
 - Grass's translational velocity is measured using a built-in motor encoder on the left leg and is sent to the ESP using wires integrated into the leg.

- The velocity data goes through a number of filters in the ESP-32 to produce a steady-enough signal as an input into the outer loop PID control system.
- **PID Controllers**
 - Two cascaded PID controllers are used in the ESP firmware to act as the foundation of the self-balancing control system for Grass.
 - The firmware intakes the inputs from the IMU (current tilt angle) and encoder (current velocity) to actuate the motors using current control and make Grass stay upright.
 - The inner loop PID is called the “balancing PID” and tries to correct Grass’s current tilt based on the difference between the IMU readings and the tilt angle setpoint. This error controls the duty cycle sent to the motors.
 - The outer loop PID is called the “velocity PID” and tries to maintain Grass’s velocity at a desired setpoint. The difference between the encoder velocity and the velocity setpoint determines the new tilt angle setpoint that Grass needs to maintain to move a certain velocity. For stationary balancing, this velocity setpoint is set to zero.

3.0 Summary of Functional Requirements

Functional Requirement	% Effort
#1: Mechanical Robot Design and Construction	20
#2: Wireless User Control	15
#3: PID Motor Control Algorithm	40
#4: Tilt Estimation and Data Processing Firmware	20
#5: Circuit Design	5

The project, Grass, consists of five major functional components. The project highlights the application of control theory and firmware to come to fruition, which cover three of the five functional requirements: Wireless User Control, PID Motor Control Algorithm, Tilt Estimation and Data Processing Firmware. These leave the mechanical and electrical design as the other two FR's: Mechanical Robot Design and Construction and Circuit Design.



Purpose and Scope of each Functional Component:

- FC#1: Mechanical Robot Design and Construction
 - **Purpose:** Creating a 3D printable body that can house all electrical components and provide protections to ensure the safety of the electrical components and its surroundings during a collision

- **Scope:** CAD and 3D Print of Chassis (Main Body) & Legs (with Motor Mounts)
- FC#2: Wireless User Control
 - **Purpose:** Creating a Bluetooth Low Energy (BLE) connection between the onboard ESP-Board and PC to allow a user to send instructions wirelessly to the robot
 - **Scope:** C# UI in WinForms (Virtual Controller) and ESP-Board firmware to set up bluetooth connection with communication in the form of serial packets (data type: strings)
- FC#3: PID Motor Control Algorithm
 - **Purpose:** Creating a control algorithm that controls the wheel velocities to self-balance the robot using inputs from the sensors (IMU and Encoder) and wireless user control
 - **Scope:** ESP-Board firmware of self-balancing control algorithm (two cascaded PIDs)
- FC#4: Data Processing Firmware
 - **Purpose:** Recording and processing information from onboard sensors: accelerometer, encoders, and gyroscope for inputs into the control algorithm
 - **Scope:** ESP-Board firmware to derive tilt angle (accelerometer & gyroscope) and wheel velocities (encoders)
- FC#5: Circuit Design
 - **Purpose:** Designing and building a circuit to provide the necessary power to each component, connect sensors and MCU together, and ensure wire management for organization and safety.
 - **Scope:** Designing circuit diagram and electrical connections between the battery, motor controller, sensors, and ESP-Board (MCU)

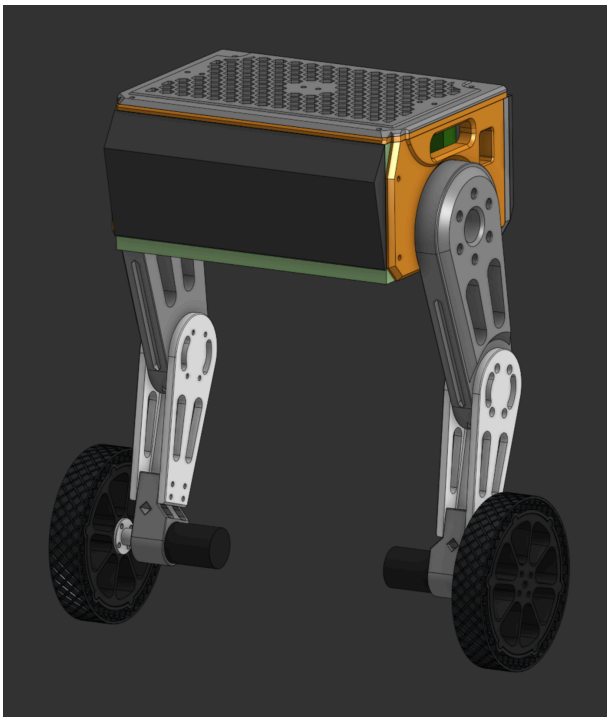
4.0 FR #1: Mechanical Design and Construction

4.1 Approach and Design

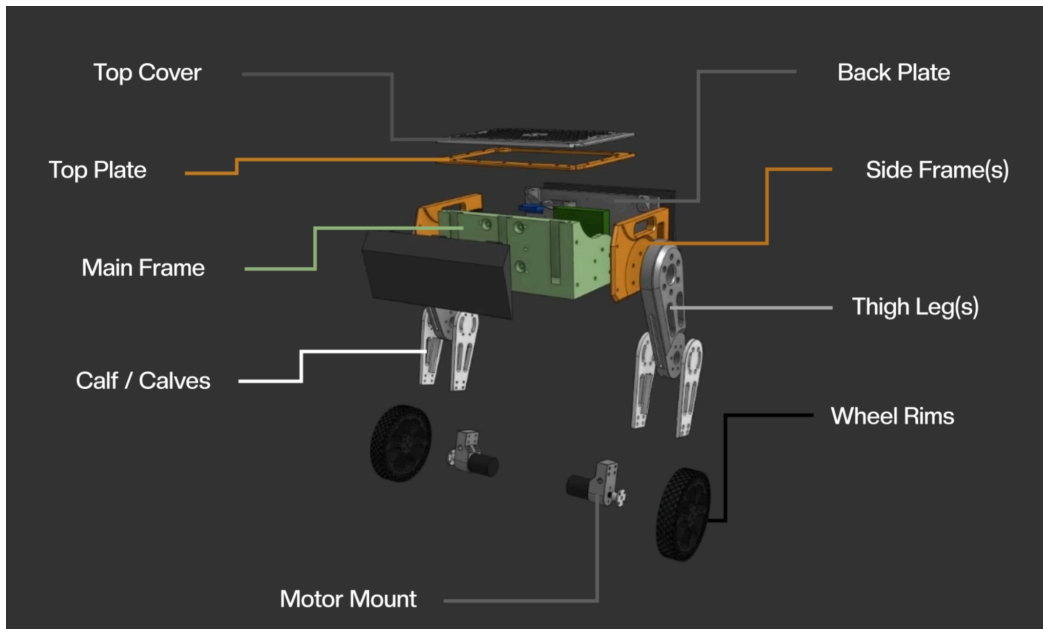
The mechanical robot design includes:

- Chassis: Main body of the robot that contains the MCU, battery, and other electrical components
- Legs: Holds the main body of the robot above the ground and houses the motors and encoders for the wheels
- Wheels: Mounts onto the motors and allows the robot to move

The objective of this functional component is to design and construct a 3D printable physical frame and wheels that can safely house any needed electrical components, protecting it from collisions and falls. This includes hiding loose wires, properly mounting the LiPo battery and motors, and adding bumpers or protections from external collisions for the safety of the robot and its surroundings.

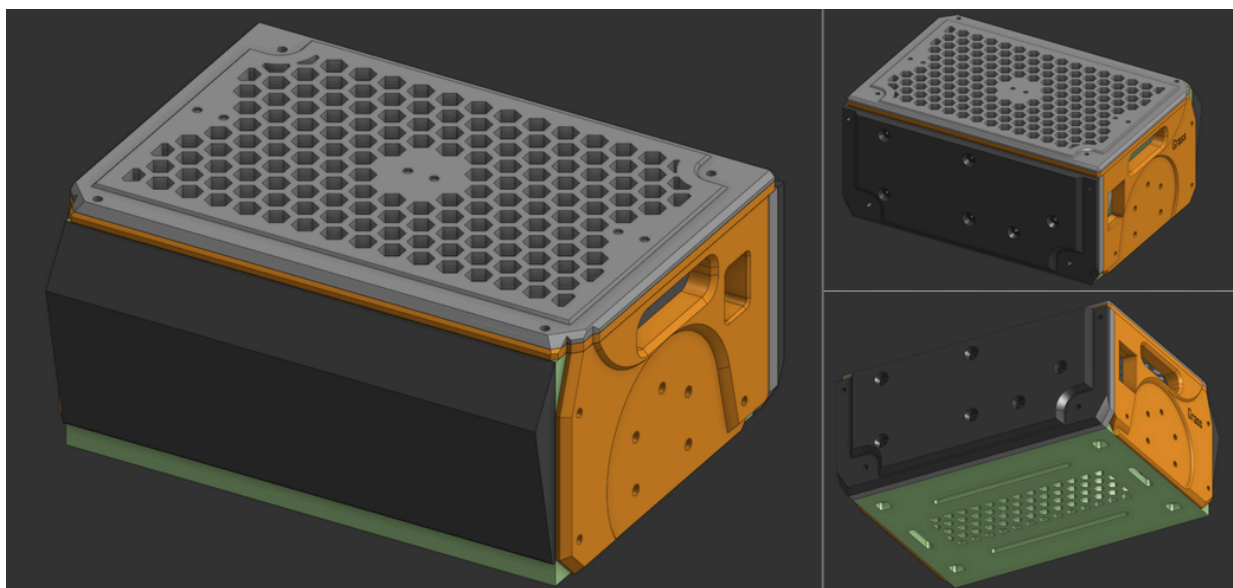


Grass's Assembly (Virtual and Physical)



4.1.1 Chassis Design

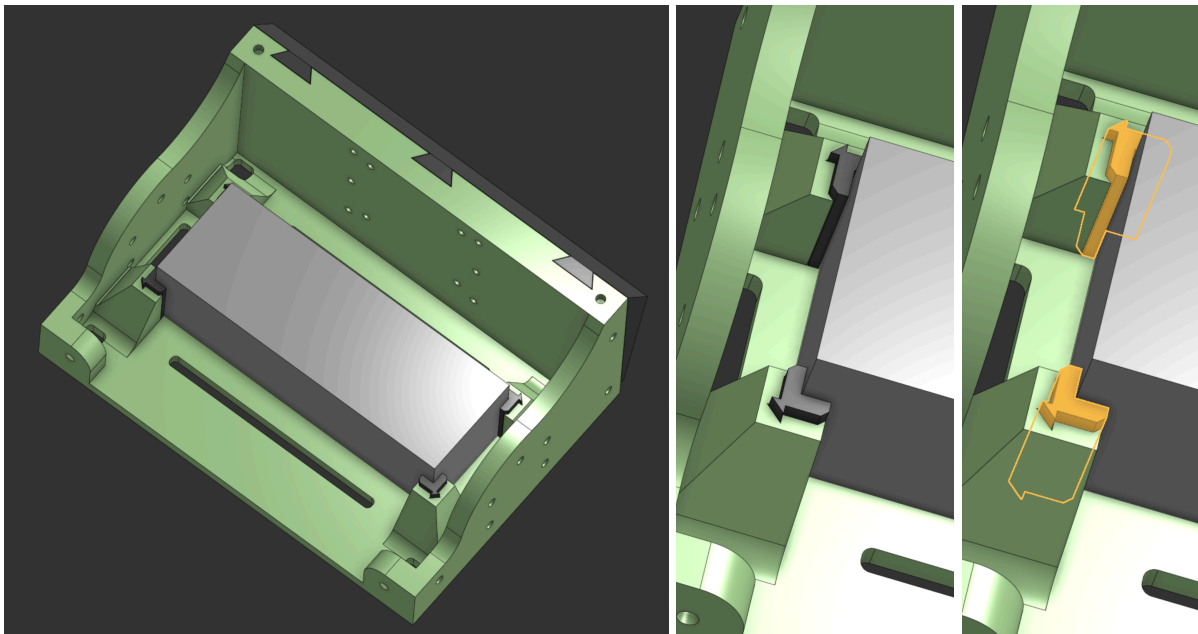
The chassis is designed in 9 main components to ensure safety, easy access, manufacturability, proper ventilation, vibration/collision protection, cross-talk prevention, and modularity. Since the chassis acts as the point mass of the inverted pendulum, there is also great consideration in the design on where the centre of mass is placed and ensuring that weight is approximately balanced between the front and back of the chassis. The overall chassis is designed in OnShape.



Chassis Components:

1. [PLA] Main Frame
2. [PLA] Side Frame Left
3. [PLA] Side Frame Right
4. [PLA] Back Plate
5. [PLA] Top Plate Outer Frame
6. [PLA] Top Plate Hat
7. [TPU] Front Bumper
8. [TPU] Back Bumper
9. [TPU] Battery Tabs

The frame is constructed of PLA which is rigid plastic and allows for a strong, yet light, foundation for Grass. TPU is a rubbery plastic which is used for the bumpers and battery tabs. The bumpers are for external protection and collisions, while the TPU battery tabs protect the battery internally from shifting the centre of mass or colliding with the other components.



Grass Main Frame - TPU Battery Tabs (Highlighted)

In the chassis, there are three main connection types. Dovetails for removable/pressfit components, 4mm holes for heated inserts, and 3.2mm holes as M3 through holes. There are also multiple holes added for easy access, hexagonal vents to view components and for ventilation, and a slot for a battery velcro strap, if required.

4.1.2 Leg Design

Each of Grass's legs are designed in four components to ensure a lightweight, structurally rigid design that can mount the motor and internal paths for wires to feed and hide the wires into. The knee joint design is mainly for aesthetics.



Leg Components:

- 10. [PLA] Thigh(s)
- 11. [PLA] Calf Inner Side
- 12. [PLA] Calf Outer Side
- 13. [PLA] Motor Mount

4.1.3 Wheel Design

The wheels are designed to use a metal flange connector to mount onto the shaft of the motor. The wheel rims are made of PLA and the wheel tire is made out of TPU. The design of the wheel rim is based on aesthetics, strength, and weight-savings. The wheels are made of TPU and have treads to ensure grip and ensure smooth movement on uneven ground.





Wheel Components:

- 14. **[PLA]** Wheel Rim
- 15. **[TPU]** Wheel Tire

4.2 Inputs and Outputs

The inputs for the construction of the robot are the maximum torque/speed of the motors and the dimensions of all the components. With the maximum torque, the safe height of the robot can be estimated. With the maximum speed of the motor, the approximate radius of the wheels required to cover balance corrections can be estimated. The output of this FC is the frame, legs, and housings for all the included components, hiding wires where necessary.

4.3 Parameters

The main parameters of the mechanical design are the height between the centre of the wheels and the centre of mass (arm of the pendulum) and the wheel radius. The higher the centre of mass of the robot can be without exceeding the torque limit of the motors, the easier it is to balance. Whereas, the wheel radius determines the torque load of the motor, amount of distance traveled with a certain degree of rotation, and impact of backlash from the motors.

Some more parameters are the printability of the chassis and legs as there may be issues with printing overhangs, small details, and the material that the parts are made of.

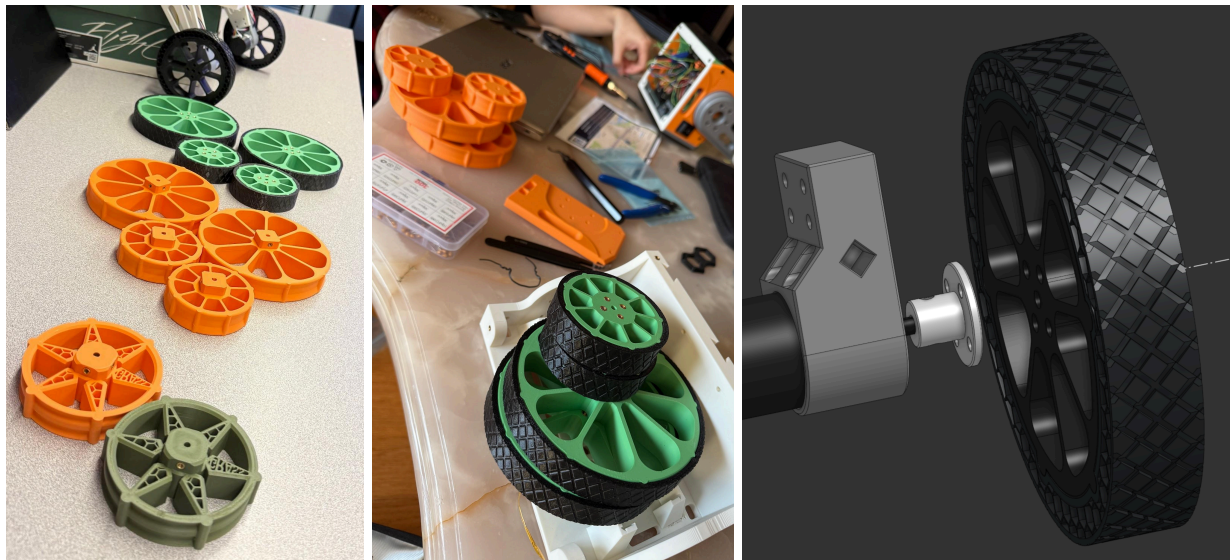
4.4 Testing and Results

The design of Grass went through multiple iterations due to damages found through testing or tuning of the robot. There were two major design changes, as a result of testing, that affected the overall design: wheels and mitigating crosstalk.

4.4.1 Wheel Changes

The most time-consuming design change is the alterations to the wheels, notably the shaft attachment mechanism and the wheel radius.

The initial design of the wheels had the shaft collar embedded in the design of the wheel rim, using M3 screws as set screws and a press-fit D-shaft hole. A large issue with this was that excessive torque during tuning would strip the D-shaft hole and eject the heated inserts melted into the plastic for the set screws. To upgrade the design, metal flange connectors are used instead. Although they still remove the wheels with excessive vibration, the metal flange does not strip and does not need replacement wheels each time damage occurs.

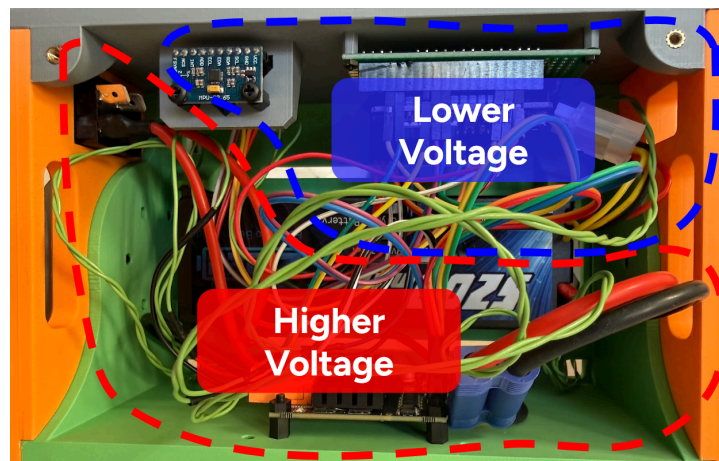


There are three wheel radius sizes to find the optimal solution for the tradeoff between torque reduction, backlash, and driving speed. Larger radius wheels increase driving speed per rotation and lower the duty cycle requirement of the motor to spin fast, but

result in larger torque requirements to spin and worse backlash effects from an improper press fit of the shaft in the gearbox of the DC motor. Smaller wheels make balancing smoother and require less torque to spin, but led to cases where Grass can no longer compensate for any imbalances or impulses due to low max speed. These tests concluded that using a medium sized wheel around 10.5cm in diameter worked well to mitigate backlash as much as possible without sacrificing too much speed.

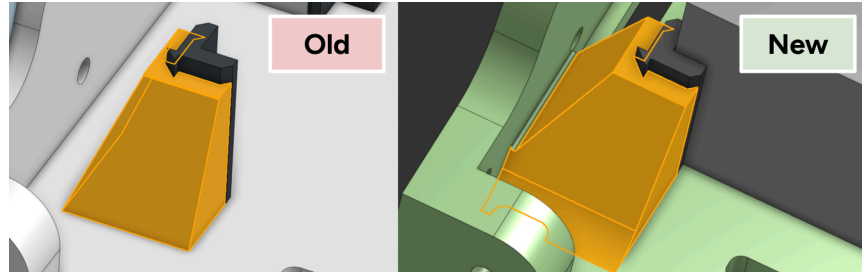
4.4.2 Crosstalk Mitigation

Crosstalk between wires was found in the early stages of the design, where additional noise was added to signal data due to close proximity between the power lines and signal lines. To mitigate this as much as possible in the mechanical design, the power wires of the motor were separated from the encoder signal wires in the leg and fed upwards on opposite sides of the chassis. Additionally, the chassis is roughly split into two halves—one side with the heavy-duty power wires (high voltage) and the other with input/output signal wires that are connected to the ESP-32 (low voltage).



4.4.3 Additional Fix - Chassis Battery Mount

A smaller fix that was implemented in the design was adding rigidity to the battery mount section of the main frame. During testing, the multiple falls and impulse tests on Grass led to a section of the battery mount falling off, so the main frame was reprinted to provide a stronger structure and better fitting TPU tabs for a tighter press fit.



5.0 FR #2: Wireless User Control

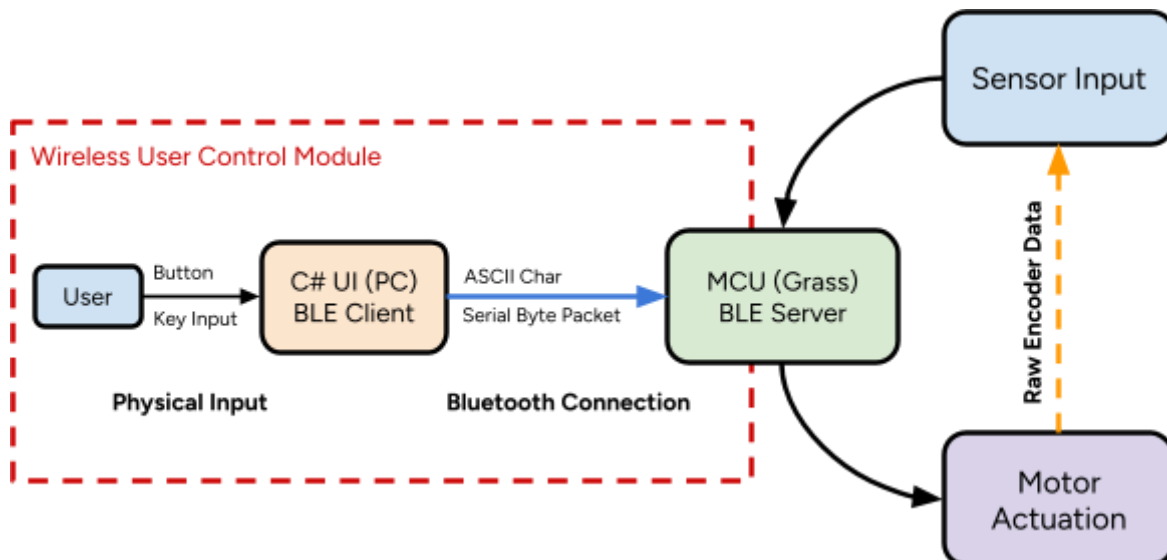
5.1 Approach and Design

The objective for the Wireless User Control module is creating a bluetooth low energy (BLE) connection between the onboard ESP-Board inside Grass's Chassis and the PC which runs a C# program that sends over command instructions. This BLE connection allows for short-ranged wireless control and tuning of Grass.

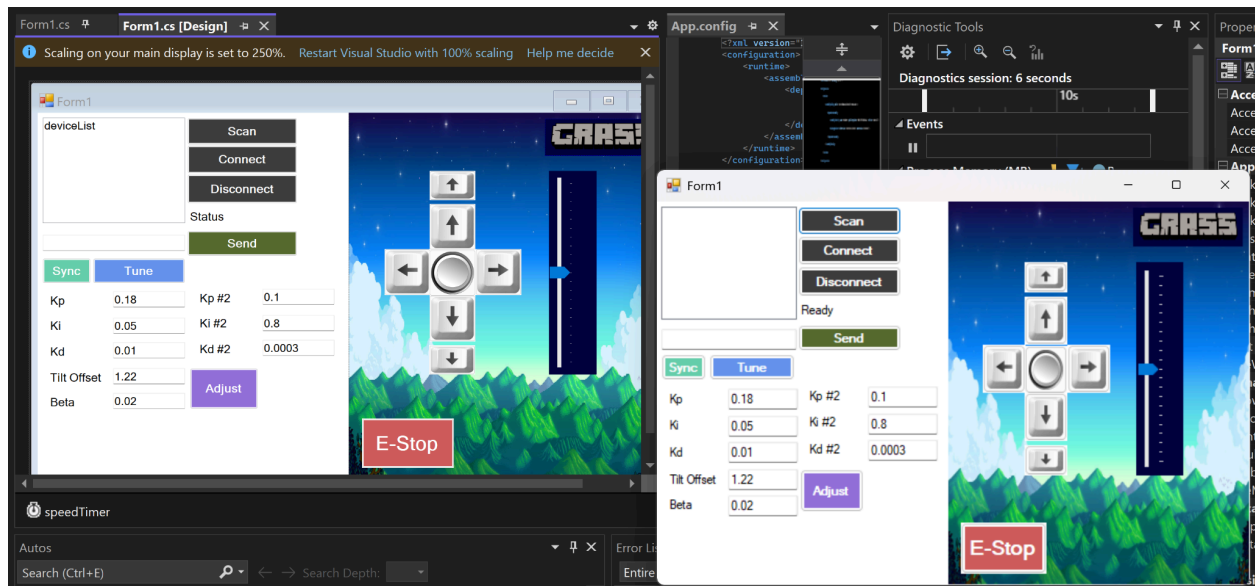
The Wireless User Control module can be split in two core components:

- C# WinForms UI on the PC that allows for bluetooth connection and sending command characters or serial byte packets based off of button/key inputs
- Arduino Firmware on the ESP-Board that sets up the board as the BLE Server to receive commands from the PC which will act as the BLE Client.

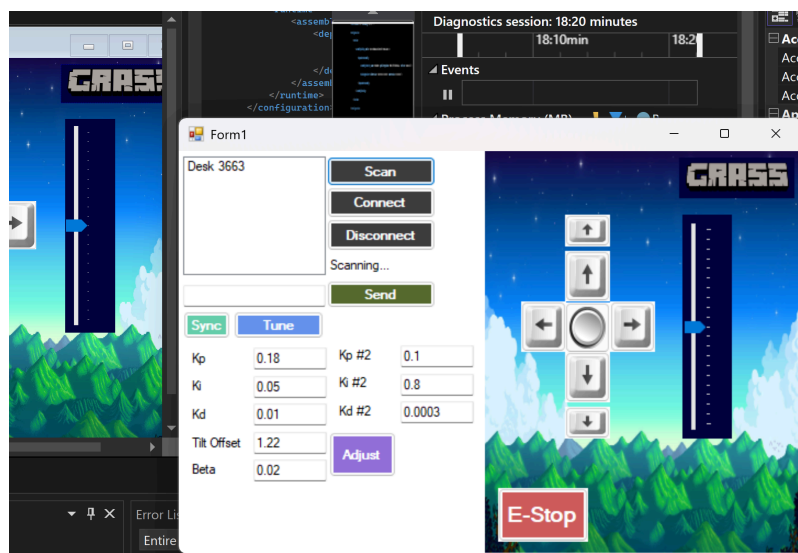
The diagram below depicts a basic overview of the Wireless User Control module.



The UI design incorporates a variety of buttons and text fields for ease-of-use and testing. The left half is used for setting up the Bluetooth connection, manual sending of custom strings, or tuning. The right half has buttons for turning and pre-set forward and backwards movement, the slider is for variable directional speed control of the motors (sends speeds of -10 to 10). The full code for the C# program is in Appendix A.

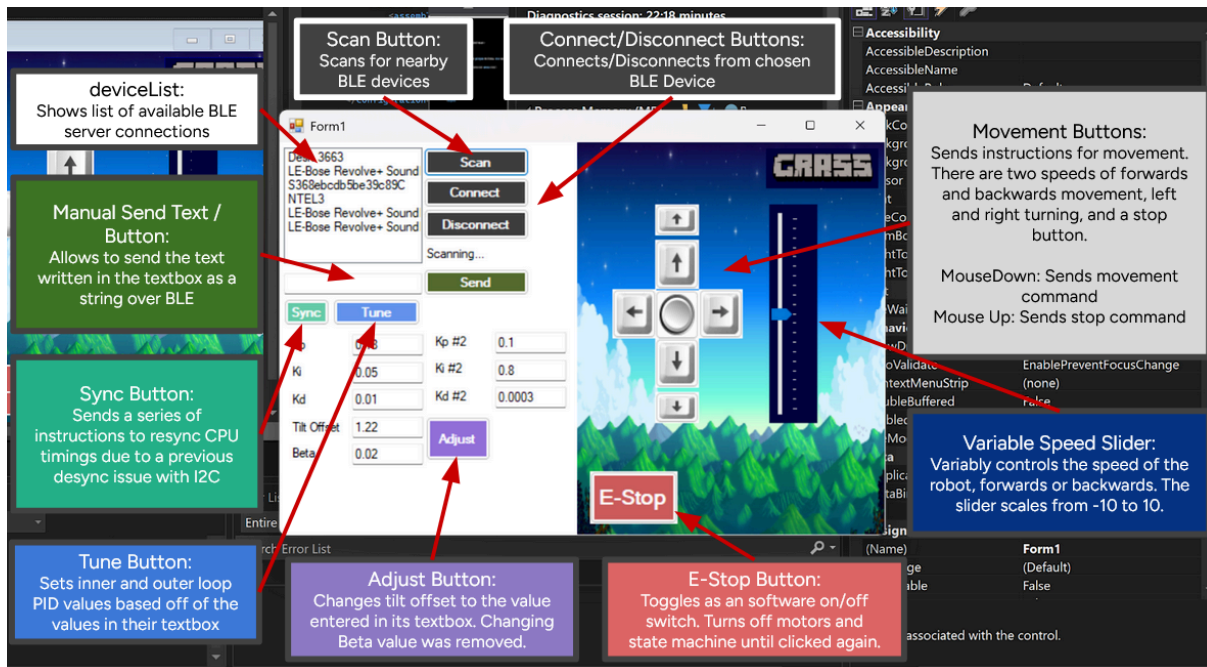


By clicking the scan button, a list of nearby bluetooth devices are listed in the “deviceList” which can be clicked.



Once a device is chosen, clicking the connect button pairs the PC to that device.

There is also built-in “status label” that changes to display when the device is connected, disconnected, what tune/control values are sent, etc. A variety of other functions are added into the UI and are displayed on the screenshot below.



The most important aspects of the UI for operation are the scan, connect, and movement buttons.

5.2 Inputs and Outputs

The main purpose of the wireless user control module is to bridge the connection between the user and the robot.

PC (C# WinForms Program) submodule

The user provides a physical input into the PC through a virtual controller created in C#. Depending on the type of input, the C# program processes the input into an instruction in the form of a string data type utilizing serial data packets. Then, over the BLE connection, the PC outputs the instruction to Grass's MCU (ESP-Board).

ESP-Board (Arduino Firmware & BLE connection) submodule

The ESP-Board receives the instructions from the PC and converts the instructions into different actions using a state machine. An example instruction is “255 10 10” which uses the command/starting byte “255” to determine what the values of the two 10's affect, whether they change Grass's velocity, turning, or tuning.

Bluetooth Low Energy Connection

The ESP-Board acts as the BLE Server, which allows the ESP to advertise itself as a connectable robot. The PC acts as the BLE Client, which scans the available services and connects to the ESP-Board over BLE, allowing it to read characteristics of the ESP-Board and send commands to it.

5.3 Parameters

The main parameters for this module are the serial byte packet structured instructions that are sent over the BLE connection. The starting byte determines the parameters that the numbers following the starting byte will affect. To do this, the ESP-32 parses through the numbers received and inputs them into a state machine. As the data type of the information being sent is a string and the infrastructure only uses a packet structure for simplicity, there is a larger variety of instructions that can be sent beyond a series of numbers.

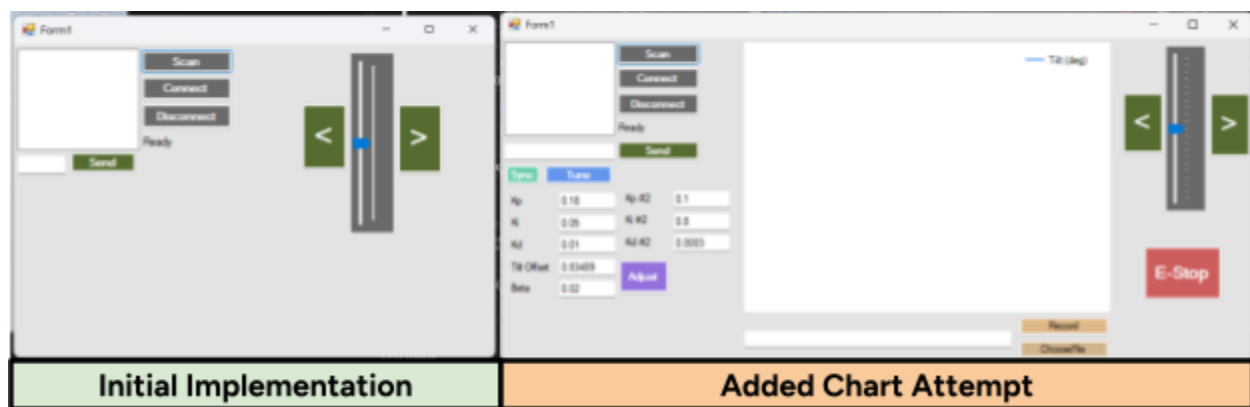
Here is a list of the starting bytes and correlated instructions that have been implemented to control/tune Grass. The names in red are implemented as the numerical value of those parameters (e.g. "255 **Kp Ki Kd**" → 255 0.5 22 5 (sent) → **Kp** = 0.5, **Ki** = 22, **Kd** = 5):

- "255 **Kp Ki Kd**"
 - Tunes the inner balancing PID values of Grass
- "252 **Kp2 Ki2 Kd2**"
 - Tunes the outer velocity PID values of Grass
- "253 **Beta**"
 - Now removed. Previously tuned a filter constant for the inner PID.
- "254 **Offset**"
 - Adds an offset to the tilt angle setpoint to accommodate for sensor drift.
- "250 **Dir Multiplier**"
 - Commands Grass to turn. Dir = 1 is to turn right. Dir = -1 is to turn left. The multiplier sets the speed of the non-encoder leg to Dir x Multiplier.
- "251 **Speed**"
 - Commands Grass to move forwards (positive number) or backwards (negative number).
- "123 234 69"
 - Instruction for E-Stop. This combination of numbers shuts down the motors and acts as a software on/off switch in case of a safety hazard.

Sending this instruction again disables the E-Stop lock and allows Grass to start back up again.

5.4 Testing and Results

The wireless user control module was tested by pairing the C# UI on one laptop to the ESP-32, then outputting the reading on another laptop that is plugged into the ESP-32 using print statements and the serial monitor on the Arduino IDE. At this stage, the UI was kept simple and did not incorporate multiple buttons to shortcut sending, as shown below on the left. Rather, the UI had a text field to send custom strings and a slider which sent a specific instruction every 100ms while being clicked on.



Once communication was confirmed, tests on the range of the bluetooth connection were conducted. The proximity test involved moving away from the ESP-32 a few meters at a time, seeing if the serial monitor can read the correct values sent and whether there was any latency on the receiving end. The test concluded that the BLE connection can be maintained from one corner of the instrumentation room to the other with no latency, even with obstacles blocking the line of sight between the PC and the ESP.

The bluetooth connection and proximity tests have been continually tested with each iteration of the UI to ensure connection and safe operation of the robot. Another addition that was attempted is adding a chart to the C# UI to read the signal data wirelessly to assist with tuning and modelling the system. Adding a chart, however, overloads the capabilities of BLE communication and the ESP-32's processing. It led to large latencies between sent instructions and received outputs or loss of communicated data, making Grass unsafe to operate and difficult to communicate with. Therefore, the chart was removed and bluetooth sending was simplified to ensure smooth and fast control.

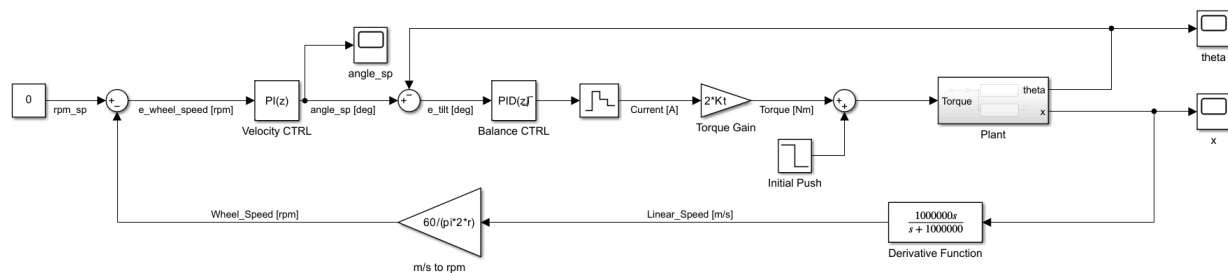
6.0 FR #3: PID Motor Control Algorithm

6.1 Approach and Design

The goal of the PID Motor Control Algorithm is to convert processed data from the Data Processing Firmware into instructions for the motors to balance the robot.

The robot will be controlled with cascaded control with two PIDs. The inner loop PID will control the motor current based on an input tilt angle setpoint while the outer loop PID will output a tilt angle setpoint based on a motor speed setpoint. To implement this in code, the “FastPID” library will be used, which minimizes delay due to float math by computing PID outputs using integers.

Below is a block diagram of the system made in Simulink.



6.2 Inputs and Outputs

Balancing PID

The inputs to the balancing PID are a setpoint tilt angle specified by the velocity PID and the tilt angle processed by the Data Processing Firmware. The output of this PID is a value from 0 to a saturated value of 5, indicating the desired current that should go into the motor. Then, using a simple motor circuit diagram, the duty cycle required to achieve this current will be calculated, taking into account the back emf generated by the spinning motor.

Velocity PID

The velocity PID receives a desired velocity input from the user and the current velocity from encoder data processed by the Data Processing Firmware. It then outputs a setpoint tilt angle into the balancing PID.

6.3 Parameters

One of the most important parameters of the PID controllers is the bandwidth. In order for the robot to balance, the closed loop bandwidth must exceed the unstable dynamics of the system. This is controlled by increasing the PID loop frequency and tuning the PID coefficients.

6.4 Testing and Results

In our testing, we had multiple attempts to tune the PIDs which resulted in major losses before attempting to model the system in Simulink.

Attempt 1: Due to a motor failure, we were left with only one motor to attempt balance. In this iteration, the free spinning wheel was locked to create a pivot point for the single motor to spin around. After hours of tuning, we realized that the motor did not have enough torque to support the robot, leading to lots of lost time as we waited for a new motor to arrive.

Attempt 2: Now with both motors, we attempted to tune the inner loop PID by setting the outer loop coefficients to zero. In this iteration, we had the loop frequency set to 250Hz and only controlled the duty cycle of the motor. During this attempt, we noticed that our 3D printed wheels had begun to strip, causing us to delay our tuning as we rushed to replace the wheels.

Attempt 3: With the same conditions and newly designed wheels, we attempted the same process of guessing and testing. While we were able to achieve balance for around 5 seconds, the robot always ended up tipping over, leading us to believe that the robot was unable to cover enough ground with our current wheel size. In addition, the wheels had once again stripped. We also noticed backlash in our wheels due to imperfections in the motor's gearbox, which led us to test multiple different wheel sizes to find a balance between linear speed and backlash.

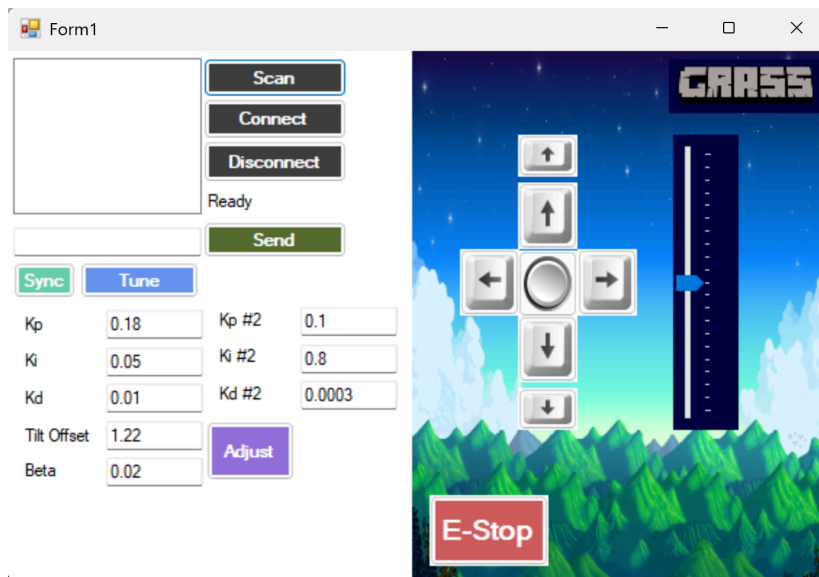
Attempt 4: With new variations of wheels, a new loop speed of 500Hz, and metal wheel flanges, we attempted to tune the PID with the outer loop active, this time with current control rather than voltage control. After a few minutes of tuning, we smelled one of the motors overheating, which caused us to halt testing and inspect the motor. In our inspection, we found that the overheating of the motor resulted in melting plastic within the motor which was then broken into small pieces as the plastic hit the rotor.

In between attempts: With only one week left, we decided to model the system on Simulink thanks to Kai Shang's advice. During this time, the motors were run slowly to estimate the motor's transfer function, which resulted in one of the motor's brushes burning out. This caused us to rush to order new replacement orders, which thankfully arrived two days later.

Attempt 5: Finally, with a fully functional Simulink model and new motors, we were able to tune the PIDs using our original wheel size while limiting the maximum duty cycle, as the motor overheating was a result of current spikes due to high-speed direction changes during balancing. The decision was made after PID tuning in the simulation, as we realized that high torque was unnecessary during balance. Additionally, monitoring the output duty cycle on the previous test revealed that the duty cycle was changing rapidly from 100% to -100%.

After implementing these changes, the robot was finally able to balance, but suffered from tilt bias which was strangely unaffected by the outer loop PID. This led us to implement a tilt setpoint bias to manually set the balancing zero position. This tilt bias is assumed to be due to insufficient gyroscope calibration, which changes whenever the robot is rebooted. While the robot was able to stay balanced within an area, we found that the robot was constantly in motion due to backlash in the wheels. Finally, we tested BLE control of the velocity setpoint and found that the robot was able to move forwards and backwards in an arc due to imbalances in the wheels.

The final parameters can be seen in the UI snapshot below.



Another significant problem we came across were limitations in the loop speed. Initially, the code was run on a single CPU using float math for the PID loops, which resulted in spikes in calculation delays, causing delays in the PID loop and losing balance. This was then solved by using multiprocessing and “FastPID” integer math, putting PID calculations in one CPU, and tilt estimation with BLE processing in the other, allowing for higher loop speeds.

7.0 FR #4: Tilt Estimation and Data Processing Firmware

7.1 Approach and Design

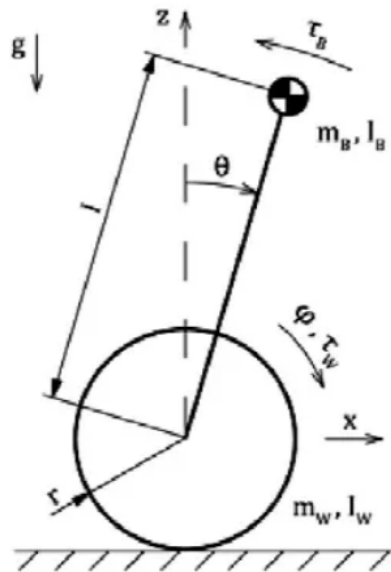
The objective of this functional requirement is to obtain and process data from the onboard IMU and encoder into a usable form for the PID motor control algorithm.

The raw IMU data will be obtained using the SPI communication protocol and converted to units of g and deg/s using the scaling factors in the datasheet. From there, the “MadgwickAHRS” library will be used in its 6-input variation to obtain an estimation of the robot’s tilt angle in degrees. To ensure a smooth signal, the IMU’s onboard low pass filter will be configured.

To get the wheel velocity in RPM, the number of edges on both phases of the quadrature encoder in a specified time frame will be counted using the

“ESP32Encoder” library, which uses the onboard hardware pulse counter. This value will then be converted to RPM by using the wheel radius and the encoder resolution. Then, to eliminate spikes in the signal, a double complementary filter will be deployed while maintaining a high bandwidth.

Below is the free body diagram of the robot, with theta identifying the tilt angle definition.



7.2 Inputs and Outputs

Inputs	Outputs
External Acceleration	Tilt Angle (deg)
Motor Rotation	Wheel Velocity (RPM)

7.3 Parameters

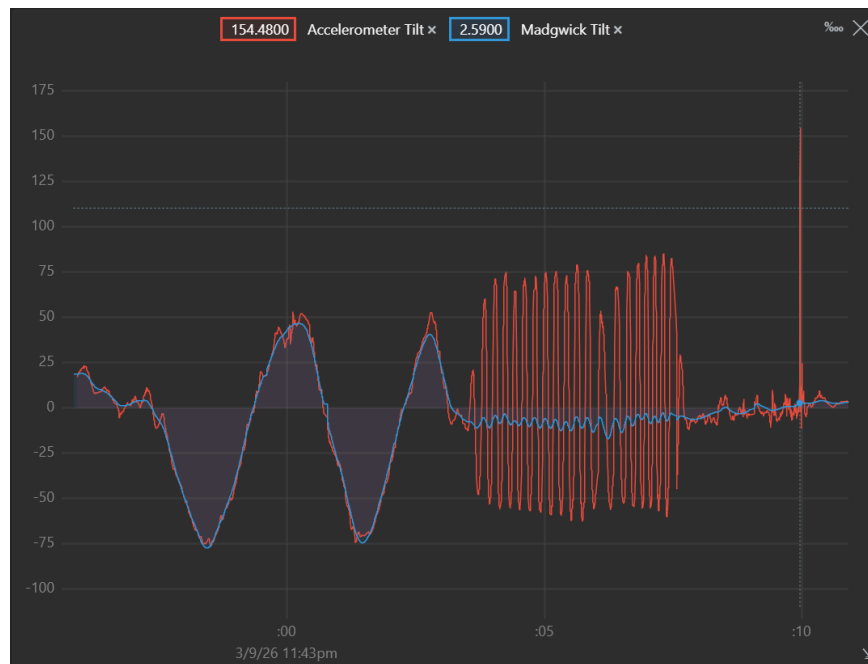
Module	Parameter	Description
MPU-6500 (IMU)	Onboard Low-Pass Filter	Cutoff frequency can be varied by writing to registers
Madgwick Filter	Beta Value	Increasing the filter's beta value adds higher weight to the accelerometer while decreasing adds higher weight to gyroscope

Complementary Filters	Alpha Value	Increasing the filter's alpha value makes the signal more smooth, but add more delay
Tilt Estimation	Loop Frequency	To minimize delay, this value must be greater than the PID loop value.

7.4 Testing and Results

During our IMU data tests, we initially used a communication protocol called I2C which caused many issues that delayed our progress. Some of these issues were the desyncing of the tilt estimation when an impulse was applied, generating a drifting tilt offset, and the sudden shutdown of the IMU data flow. While we are still unsure of the reason for desyncing, adding printouts to the code to monitor the shutdown error led us to believe that noise from the motors were corrupting our data. This led us to change our communication protocol to SPI, a protocol that is more resistant to noise, which solved both issues.

To help tune the beta value, we compared the tilt angle from the filter with a tilt angle calculated with the measured accelerations, which would be accurate with minimal impulses. Tuning the value for maximum smoothness and bandwidth led us to an accelerometer gain value of 0.02 and a low pass filter cut-off frequency of 98Hz. The image below shows the effectiveness of the filter's tracing and resistance against external accelerations.



Similarly for the alpha value, we monitored the velocity and visually observed noise spikes and the delay when stopping the motor, which led us to an alpha value of 0.2.

Finally, our tilt estimation frequency was chosen to be 1 kHz which was double the frequency of the PID loop.

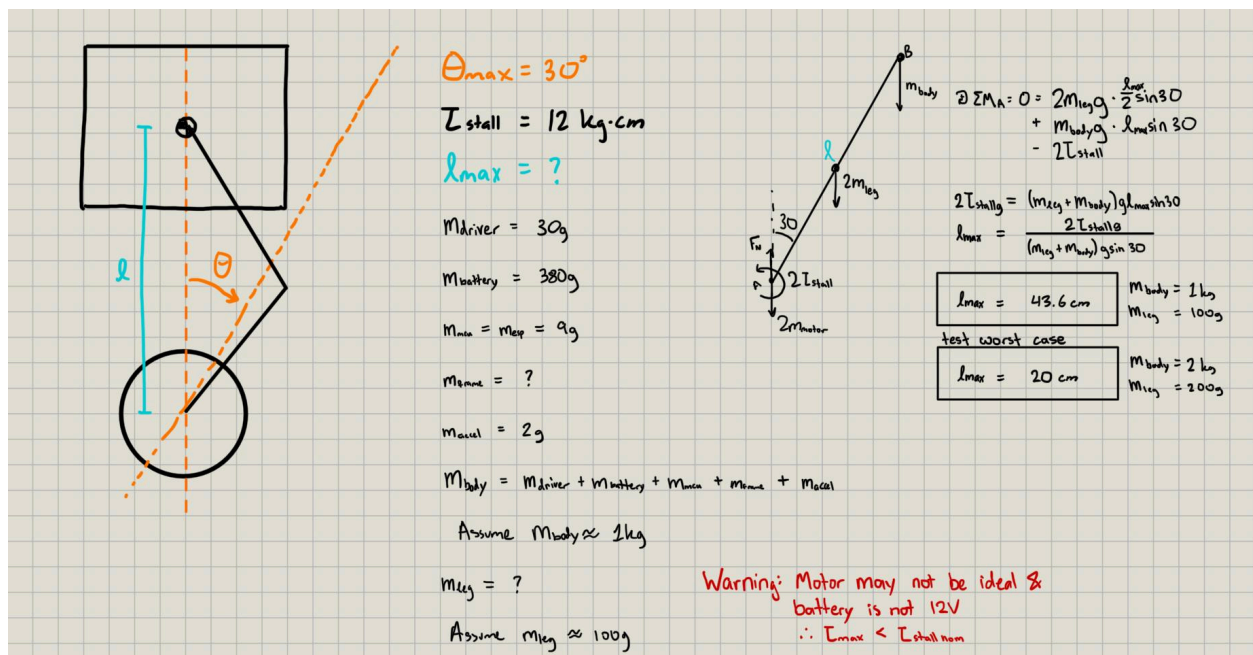
8.0 FR #5: Circuit Design

8.1 Approach and Design

The objective of the Circuit Design is to select and plan out the necessary electronics to ensure that all components receive sufficient power while making sure each component has the necessary parameters to execute the project.

The components will be selected by calculating the maximum required torque when the robot is tilted 30 degrees. This torque value will then be used to select a motor. From there, the specifications in the motor datasheet will be used to identify the required components, such as buck converters and battery voltages. The stall currents from the motor and input voltage will then be used to choose a motor controller. For the IMU, any device with accelerometer and gyroscope data will be sufficient and a device with bluetooth capabilities and high clock speed will be chosen for the MCU.

The calculation below shows the estimated maximum length from the wheel motors to the centre of gravity of the body, assuming a maximum tilt angle of 30 degrees and a stall torque corresponding to the FIT0493 motor.



$\theta_{max} = 30^\circ$
 $I_{stall} = 12 \text{ kg}\cdot\text{cm}$
 $l_{max} = ?$

$m_{driver} = 30g$
 $m_{battery} = 380g$
 $m_{min} = m_{max} = 9g$
 $m_{motor} = ?$
 $m_{axial} = 2g$
 $m_{body} = m_{driver} + m_{battery} + m_{min} + m_{max} + m_{axial}$
 Assume $m_{body} \approx 1kg$
 $m_{leg} = ?$
 Assume $m_{leg} \approx 100g$

Diagram 1: A rectangle representing the robot body is tilted at an angle θ from a vertical dashed line. A vertical line of length l represents the distance from the pivot to the center of gravity.

Diagram 2: A free-body diagram of the robot arm. It shows a pivot at the bottom with a counter-clockwise torque $2I_{stall}$. The arm has a mass $2m_{leg}$ at its midpoint and a body mass m_{body} at the end. The arm is tilted at 30° from the vertical. A force F_{cg} acts at the center of gravity.

Equations:

$$\sum \tau_{MA} = 0 = 2m_{leg}g \cdot \frac{l_{max}}{2} \sin 30^\circ + m_{body}g \cdot l_{max} \sin 30^\circ - 2I_{stall}$$

$$2I_{stall}g = (m_{leg} + m_{body})g l_{max} \sin 30^\circ$$

$$l_{max} = \frac{2I_{stall}}{(m_{leg} + m_{body}) \sin 30^\circ}$$

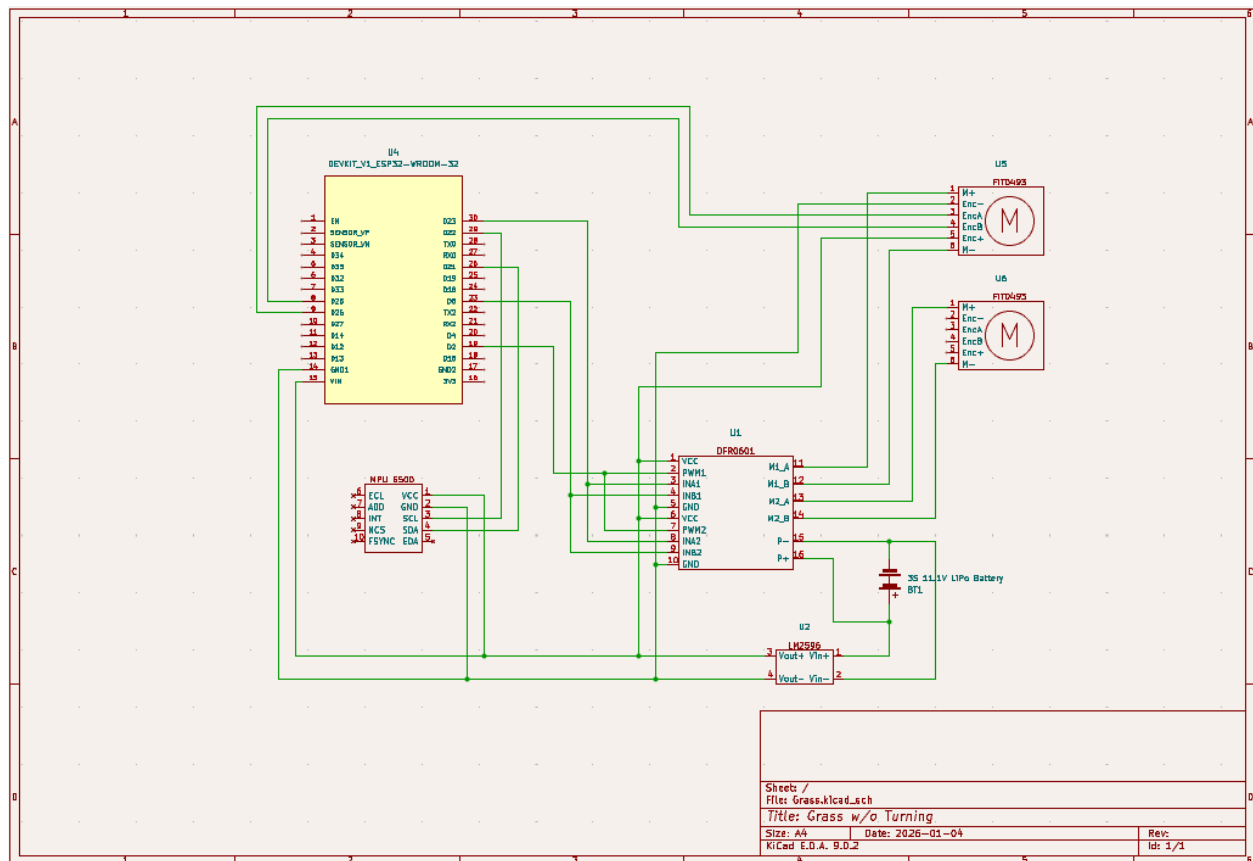
$l_{max} = 43.6 \text{ cm}$	$m_{body} = 1kg$ $m_{leg} = 100g$
-----------------------------	--------------------------------------

test worst case

$l_{max} = 20 \text{ cm}$	$m_{body} = 2kg$ $m_{leg} = 200g$
---------------------------	--------------------------------------

Warning: Motor may not be ideal & battery is not 12V
 $\therefore I_{max} < I_{stall nom}$

The circuit diagram below shows the expected wiring without the consideration of turning.



8.2 Inputs and Outputs

The input for the circuit diagram is the desired robot height (roughly 20cm). From this desired height, the key parameters are obtained using component datasheets.

The output for this module is a functioning circuit with ratings that appropriately support two motors with sufficient torque.

8.3 Parameters

The key parameters for the circuit diagram include the voltage and current ratings of each component. A buck converter is necessary since most components have a 3.3 V rating while the motor controller has a 12V rating. Additionally, the current drawn from the motors is important to note when choosing a battery.

Another parameter is the clock frequency of the MCU as the sample frequency must be greater than the tilt estimation bandwidth for the robot to balance.

8.4 Testing and Results

After identifying the necessary electronics, the following components were selected:

Required Module	Chosen Module	Key Specifications
Battery	1x 3s 11.1V LiPo Battery 5200 mAH	• ~12V Output • >12A Output • >20min runtime
Buck Converter	1x LM2596 Buck Converter	• Rated 3A • >5V Rated Output Voltage • >12V Rated Input Voltage
DC Motor and Encoder	2x FIT0493 Brushed DC Motor and Encoder	• Rated 12V • 12kgcm Stall Torque • High Resolution Encoder
IMU	1x MPU-6500	• Accelerometer and Gyroscope • Onboard Low-Pass Filter
Motor Controller	1X DFR0601 12A Dual Channel Motor Controller	• 12A Rating • Dual Channel • 12V Input Voltage
MCU	ESP32 WROOM Module	• Wifi & Bluetooth Chip • SPI Pins • 240 MHz Clock Speed • Dual Core Processor • Many GPIO Pins • 5V Input Voltage

After building the previously shown circuit and running the code, the circuit worked seamlessly.

9.0 System Evaluation

9.1 Self-Balancing Test

The self-balancing test evaluates whether Grass can sustain balance over a prolonged period of time. Being able to self-balance is the core function of this robot, therefore this evaluates the internal self-balancing control system and overall construction of the robot.

The specific metric of success for this test will be whether the robot can balance without external input for at least 5 minutes.

By testing the self-balance algorithm over a prolonged period of time, we can also observe whether the PID tuning can be optimized. Therefore, a secondary objective of this test is to minimize the compensation movement done to balance the robot by physically measuring the distance the robot oscillates about its equilibrium (balanced) position and fine tuning the PIDs, if time permits.

After conducting the tests, we were able to verify that Grass can stay balanced for over 5 minutes. In addition, Grass is able to stay upright while oscillating very slightly (<5mm) due to motor backlash. As previously mentioned, Grass is unable to stay perfectly still for the same reason.

9.2 Movement Test

The movement test is a simple observation to evaluate whether the robot can move forwards and backwards while self-balancing. Therefore, the metric of success is whether Grass can move at least one meter forward and one meter backward without falling. Although the test is simple, the control algorithm required for this movement is complex and may require multiple iterations.

Grass was able to comfortably travel forwards and backwards for as long as the user desired, though imbalances in the motors cause Grass to travel in an arc rather than in a straight line.

9.3 Bluetooth Proximity Test

The purpose of the bluetooth proximity test is to check the range of the bluetooth connection when the ESP-Board is in an enclosure and ensure Grass can safely operate when disconnected.

The metric of success is whether Grass moves to idle self-balancing when disconnected from the PC. Measurements on the bluetooth proximity will be used to allow for further iterations of this test, and to indicate what proximity the user must be to the robot. Failure of this test would be the fall of Grass when disconnected from the PC.

Two versions of this test will be conducted. One where the user moves away from the robot, and another where the robot is commanded to move forward and exceeds the bluetooth proximity distance.

As previously mentioned, Grass was able to stay connected to the computer from across the instrumentation lab. In addition, disconnecting from Grass resulted in stable balancing. This led us to automatically pass this test as the bluetooth proximity far exceeded our expectations.

9.4 Impulse Recovery Test

The impulse recovery test is a more extreme version of the self-balancing test. Rather than testing whether Grass can balance for a prolonged period of time, this test is to see if Grass can continue to self-balance after an impulse/collision/nudge.

The test method will involve pushing grass with a variety of strengths from a light nudge to a hard shove (relatively determined). The test will be considered a success if Grass can continue to self-balance after a light push. This will ensure that any mistaken collisions with Grass will not lead to catastrophic failure. Increasing the strength of the impulse is to further evaluate the safe limit that Grass can endure and the robustness of the balancing control algorithm.

Grass was able to withstand light playful pushes, quickly recovering and staying upright. Collisions with walls or other obstacles also did not result in falling, allowing Grass to pass this test. We found that Grass would fail to balance when impulsed with a medium strength shove.

9.5 Turning Test [Reach Goal]

To evaluate its turning capability, simple turns will be observed. While Grass self-balances, we can test its ability to turn 360 degrees for both directions (left and right). If this simple test is completed, turning can be considered a success.

By biasing one motor given a discrete input, we were able to achieve slow turning. While the turning is slow, the test was a success as Grass was able to turn 360 degrees in both directions.

10.0 Reflections

10.1 Dominic

What worked and what didn't work? Why? What would you do differently if you could do it again?

I found that a lot of the project went according to plan, other than the motors failing. For instance, every electronic component chosen worked as intended. The circuit design had little to no iterations as it worked on the first try and the physical frame of the robot only went through one iteration. We also already had a couple of the components lying around at home, which saved us some time. I also think that previous experience with personal projects and testing helped give us some insight into the key parameters in this project, such as maximizing the centre of mass of the robot and understanding LiPo battery safety.

Some things that did not work well were issues in the firmware and with the motors. There were many unexpected issues in the firmware, such as figuring out multiprocessing, figuring out a way to measure tests with minimal printouts (since `Serial.print()` introduces delays in sample time), and figuring out how to properly measure speed in a noisy environment. With the motors, we were uneducated on the limitations of brushed DC motors in the scope of our project. We were unaware of the risk of our brushes burning out due to repeated direction changes, the fragility of the motor's brush connections, and the effect of motor backlash on our project. Another large mistake was trying to tune the PIDs visually rather than making a simulation.

If I could redo the project, I would use brushless motors to mitigate the risks of burnout and eliminate backlash by choosing motors with a higher quality gearbox. To address our firmware issues, I would choose a stronger MCU with more cores to allow for higher loop speeds and wireless data monitoring to avoid serial prints. Most importantly, if I were able to start over, I would begin with a Simulink model to help tune the PIDs and loop speeds.

Identify 3 things you learned in MECH 421-423 that you consider the most useful and why.

The three most important things I learned in this course were software development, firmware development, and the range of things we can do as mechatronics engineers.

I think that software development is very important because it allows us to bridge the gap between the user and the product. Before this course, the only form of UI development we were exposed to was SCADA, which seemed very limited. In learning C# and WinForms, I believe that I can elevate my future personal projects to be more tailored to the product and more user friendly, rather than using premade apps.

In my degree, I only had experience working with Arduino and other adjacent MCUs which was more of a hobbyist level of firmware. In working with MSPs, I was able to understand better how firmware development actually looks like in industry. I am glad to have gained some experience that could help me partially bridge the gap to industry work.

Before this course, I was left with fragments of information from previous supporting courses, such as ELEC 343, MECH 420, and other courses that focus on theory. In my opinion, UBC focuses a lot on theory, as if it prepares students more for research than preparing them for industry work. After experiencing this course, I was able to connect all the pieces of information and combine them into something interesting. I really enjoyed seeing everyone's projects and seeing how much we can actually do with the knowledge we've gained as mechatronics students.

What are some limits of your knowledge and expertise as a mechatronic engineer? Identify 3 things you would like to learn going forward. What is your strategy to acquire knowledge in these areas?

With the knowledge that I currently have, I think that I am able to create a large quantity of things that I want to design and build. Though, I am not very confident in circuit design, modeling complex systems, and using CAD software efficiently.

In the electronics portions of my projects, I always rely on prebuilt modules rather than designing a PCB and learning how to implement microchips. I also could not see the value in making a PCB when a lot of modules are available on the market. In building this project, I was able to see the value in PCB building, such as compactness and neat wiring. In my next project, I plan to build a PCB to avoid messy wiring and to improve my skills.

During this project, we were very fortunate to have been able to find the physics derivations for our project online, though I felt unfulfilled since I feel that I should have been able to derive those equations from scratch. From that resource, I was able to better understand how to model an unstable system, but am still not very confident that I can do it well. I will further develop this skill by making a simulation of my next project.

Although I was able to CAD the legs and motor housing for this project, I felt that I wasn't using the software to its fullest as SOLIDWORKS has a lot of different buttons that I do not fully understand how to use. To improve my CAD skills, I intend to take some classes on using SOLIDWORKS or some other CAD software.

10.2 Nicholas

What worked and what didn't work? Why? What would you do differently if you could do it again?

The project itself was a huge success and I am very happy that Grass managed to self-balance, be controlled, and withstand impulses. It has all the main functions that we could have hoped for in the time frame given and was a great experience to build.

The final design of Grass managed to work, but there were initial designs that we spent multiple days iterating on to get to the final point. At least on the CAD side, there were multiple changes for spacing, overall chassis size, adding clearance, or overall design overhauls that were made during the design process. However, the design of Grass itself was split into many parts for the purpose of iteration, which helped a lot when we had to switch out components continuously for "a better fit". I learned a lot about heat inserts, clearances, and design for manufacturing just by building the chassis. I also learned how important CPU management is when having computational heavy programs on a small chip. Though, apart from everything, I think the thing that surprised me the most was how easy it is to read nearby bluetooth devices and possibly connect to them.

Regardless of all the changes, everything mostly went according to the schedule that we had planned and allowed us to reach a completed project in time for the demo. I think the only section of the project that took a lot of time was getting the control system working for balancing and tuning the PIDs. At first, we tried to tune manually. Grass was able to balance pretty well, but not enough to stay upright for a long period of time and it was unable to utilize the outer PID properly. That is when we decided to

model the system and get the PID values that way, which turned out way better than we were expecting.

The main thing I would change about this project would be to use BLDC motors instead of a DC gearmotor to drive the wheels. We encountered way too many problems with motor failure, noisy encoder data, excessive backlash, uneven performance between motors, etc. We thought it would be the cheaper alternative considering the small budget, but led to multiple costs to cover replacements that we would not have liked. A BLDC motor would prevent most of the wear, drifting, and backlash issues we were encountering and would be more interesting to figure out how to control. The only issue would be high starting costs and complex motor driving requirements, but they could be accounted for and would make a more sleek implementation.

If I could improve Grass, I would like to properly include different modes of turning. For example, having varying speed between the wheels to allow for turning while moving in a direction.

Identify 3 things you learned in MECH 421-423 that you consider the most useful and why.

Rather than just a technical skill, I think the most useful thing I learned in MECH 421/423 is the confidence in my own skills as a mechatronics engineer. Throughout the degree, there is so much theory that gets crammed in our brains, with many parts of it being forgotten. There has not really been a time where I could both validate and gain practical engineer skills at the same time. All the problem-solving, building, and soldering helped me learn to have more confidence in myself as an engineer and is definitely the thing I appreciate the most about the course.

I think the next most useful thing that I learned was firmware, and actually learning the integration of a mechatronics system. Although the mechatronics degree attempts to teach "each part" of a mechatronics system, it always felt separated until this course. Learning firmware also opened a new horizon of possibilities and understanding on how microcontrollers work and how we can use them for robotics. It became a proper bridge between the mechanical, hardware, and software aspects that meshed together nicely with my previous engineering knowledge.

Another thing I learned in 421/423 that is useful is C# WinForms development and bridging the connection between the MSP and the C#. And, not just learning C# as a language, but instead learning how to create a UI and seeing how web apps can be

developed from the bottom up. In a sense, it made the firmware and robotics control feel more accessible. Making the controller for Grass from scratch, learning how to connect to a microcontroller over UART, and developing that pathway between them gave such a great experience that I enjoyed and found extremely helpful for any future projects I may do.

What are some limits of your knowledge and expertise as a mechatronic engineer? Identify 3 things you would like to learn going forward. What is your strategy to acquire knowledge in these areas?

In an honest review of my skills, I do not believe I have as strong mechanical knowledge when it comes to areas like fluids or beam-bending. Nor, do I understand enough about circuit filters to understand where I should use them and filter by how much. And, I believe there is much left for me to learn about software and coding. My limits are drawn from understanding only the broad-overview of a mechatronics system and not the dive-deep specifics of one area. And, in my degree, there were not many chances to improve those skills outside of school apart from extensive CAD designing in my engineering design team.

Therefore, if I were to learn more going forward, I would like to work on those skills. I would like to dive-deeper into applying fluids and beam-bending equations through simulation, FEA, and manual calculations to create a desired product. I would like to further my software skills by learning how to code more games, useful apps, or how to process data more effectively. And, I would like to learn how to apply my current circuit knowledge to actual designs and create a PCB that I can call my own.

To gain these expertise, I believe most of the limits and “improvements” I mentioned just come with experience. By working and developing as an engineer, I will naturally gain more confidence in my abilities to create things and be more comfortable with my current knowledge/expertise. I would also like to start making other robotics projects after MECH 421/423 to supplement my experience and provide myself a motive to dive into different problems. There is so much information online to develop myself further, I just need to find the motive and direction to learn from it.

APPENDIX A - C# WinForms UI Code

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Windows.Forms;
using Windows.Devices.Bluetooth;
using Windows.Devices.Bluetooth.Advertisement;
using Windows.Devices.Bluetooth.GenericAttributeProfile;
using Windows.Foundation;
using Windows.Storage.Streams;
using static System.Net.Mime.MediaTypeNames;

namespace GrassBluetooth
{
    public partial class Form1 : Form
    {
        private BluetoothLEAdvertisementWatcher watcher;
        private BluetoothLEDevice bleDevice;

        private float Kp = 0;
        private float Ki = 0;
        private float Kd = 0;

        private float Kp2 = 0;
        private float Ki2 = 0;
        private float Kd2 = 0;

        private float Beta = 0;
        private float Offset = 0;

        // Keep discovered devices (key = BluetoothAddress, value = name)
        private Dictionary<ulong, string> foundDevices = new Dictionary<ulong,
string>();

        GattCharacteristic txChar; // The BLE characteristic we write to
        GattCharacteristic rxChar; // BLE characteristic we receive tilt from
        int tiltTimeIndex = 0;

        private bool isRecording = false;
        private string csvFilePath = null;
        private System.IO.StreamWriter csvWriter = null;
        private DateTime logStartTime;

        public Form1()
        {
            InitializeComponent();
        }
    }
}
```

```

    }

    private void Form1_Load(object sender, EventArgs e)
    {
        lblStatus.Text = "Ready";
    }

    // -----
    // SCAN BUTTON
    // -----
    private void btnScan_Click(object sender, EventArgs e)
    {
        deviceList.Items.Clear();
        foundDevices.Clear();
        lblStatus.Text = "Scanning...";

        watcher = new BluetoothLEAdvertisementWatcher
        {
            ScanningMode = BluetoothLEScanningMode.Active
        };

        watcher.Received += Watcher_Received;
        watcher.Stopped += Watcher_Stopped;

        watcher.Start();
    }

    private void Watcher_Received(BluetoothLEAdvertisementWatcher sender,
BluetoothLEAdvertisementReceivedEventArgs args)
    {
        string name = args.Advertisement.LocalName;
        if (string.IsNullOrEmpty(name)) return;

        if (!foundDevices.ContainsKey(args.BluetoothAddress))
        {
            foundDevices[args.BluetoothAddress] = name;

            Invoke((Action)(() =>
            {
                deviceList.Items.Add(name);
            }));
        }
    }

    private void Watcher_Stopped(BluetoothLEAdvertisementWatcher sender,
BluetoothLEAdvertisementWatcherStoppedEventArgs args)
    {

```

```

        Invoke((Action)(() =>
        {
            lblStatus.Text = "Scan stopped";
        }));
    }

    // -----
    // CONNECT BUTTON
    // -----
    private void btnConnect_Click(object sender, EventArgs e)
    {
        if (deviceList.SelectedIndex < 0) return;

        string selectedName = deviceList.SelectedItem.ToString();

        // Get the address of the selected device
        ulong address = foundDevices.FirstOrDefault(kv => kv.Value ==
selectedName).Key;

        if (address == 0)
        {
            lblStatus.Text = "Device not found";
            return;
        }

        lblStatus.Text = "Connecting...";

        // Use Completed handler instead of await
        IAsyncOperation<BluetoothLEDevice> op =
BluetoothLEDevice.FromBluetoothAddressAsync(address);
        op.Completed = new
AsyncOperationCompletedHandler<BluetoothLEDevice>((asyncOp, status) =>
        {
            if (status == AsyncStatus.Completed)
            {
                bleDevice = asyncOp.GetResults();
                if (bleDevice != null)
                {
                    bleDevice.ConnectionStatusChanged +=
BleDevice_ConnectionStatusChanged;
                    Invoke((Action)(() =>
                    {
                        lblStatus.Text = "Connected to " + bleDevice.Name;
                        DiscoverServices();
                    }));
                }
            }
            else

```



```

        {
            Invoke((Action)(() =>
            {
                lblStatus.Text = "Connection failed";
            }));
        }
    }
    else
    {
        Invoke((Action)(() =>
        {
            lblStatus.Text = "Connection failed";
        }));
    }
});
}

// -----
// CONNECTION STATUS CHANGED
// -----
private void BleDevice_ConnectionStatusChanged(BluetoothLEDevice sender,
object args)
{
    Invoke((Action)(() =>
    {
        lblStatus.Text = "Status: " + sender.ConnectionStatus.ToString();
    }));
}

// -----
// DISCONNECT BUTTON
// -----
private void btnDisconnect_Click(object sender, EventArgs e)
{
    if (bleDevice != null)
    {
        bleDevice.Dispose();
        bleDevice = null;
        isRecording = false;
        csvWriter?.Flush();
        lblStatus.Text = "Disconnected";
    }
}

// -----
// STOP SCAN BUTTON (optional)
// -----

```

```

private void btnStopScan_Click(object sender, EventArgs e)
{
    if (watcher != null)
    {
        watcher.Stop();
        lblStatus.Text = "Scan stopped";
    }
}

// -----
// SEND FUNCTIONS
// -----
private void DiscoverServices()
{
    var servicesOp =
bleDevice.GetGattServicesAsync(BluetoothCacheMode.Uncached);
    servicesOp.Completed = (op, status) =>
    {
        if (status != AsyncStatus.Completed) return;

        var services = op.GetResults().Services;

        foreach (var s in services)
        {
            if (s.Uuid ==
Guid.Parse("7bd8e1e7-186b-4442-bd1e-eedef0bb9139"))
            {
                var charsOp =
s.GetCharacteristicsAsync(BluetoothCacheMode.Uncached);
                charsOp.Completed = (cop, cstatus) =>
                {
                    if (cstatus != AsyncStatus.Completed) return;

                    foreach (var ch in cop.GetResults().Characteristics)
                    {
                        // TX (PC → ESP32)
                        if (ch.Uuid ==
Guid.Parse("0644d551-7b7f-4013-998d-f009450e801b"))
                        {
                            txChar = ch;
                        }
                    }

                    Invoke((Action)(() =>
                    {
                        lblStatus.Text = "BLE Ready (TX + RX)";
                    }));
                }
            }
        }
    }
}

```

```

        };
    }
}

public void SendByte(byte b)
{
    if (txChar == null) return;

    DataWriter writer = new DataWriter();
    writer.WriteByte(b);

    var op = txChar.WriteValueAsync(writer.DetachBuffer());
    op.Completed = (o, s) => { };
}

public void SendString(string text)
{
    if (txChar == null) return;

    DataWriter writer = new DataWriter();
    writer.UnicodeEncoding = UnicodeEncoding.UTF8;    // force ASCII/UTF8
    writer.WriteString(text);

    var op = txChar.WriteValueAsync(writer.DetachBuffer());
    op.Completed = (o, s) => { };
}

private void btnSend_Click(object sender, EventArgs e)
{
    if (txChar == null)
    {
        lblStatus.Text = "Not connected to BLE";
        return;
    }

    string text = textBoxSend.Text;

    if (string.IsNullOrEmpty(text))
        return;

    // If user types hex like: 0x55 or 55
    if (text.StartsWith("0x") || text.Length <= 2 && IsHex(text))
    {
        try
        {
            byte value = Convert.ToByte(text.Replace("0x", ""), 16);

```

```

        SendByte(value);
        lblStatus.Text = "Sent byte: 0x" + value.ToString("X2");
    }
    catch
    {
        lblStatus.Text = "Invalid hex byte";
    }
}
else
{
    // Otherwise send as ASCII text
    SendString(text);
    lblStatus.Text = "Sent text: " + text;
}

HandleSentText(text);
}

private bool IsHex(string text)
{
    foreach (char c in text.Replace("0x", ""))
    {
        if (!Uri.IsHexDigit(c))
            return false;
    }
    return true;
}

private void HandleSentText(string text)
{
    // Expected format: 255 Kp Ki Kd
    string[] parts = text.Trim().Split(' ');

    if (parts.Length == 4 && parts[0] == "255")
    {
        if (float.TryParse(parts[1], out float kp) &&
            float.TryParse(parts[2], out float ki) &&
            float.TryParse(parts[3], out float kd))
        {
            Kp = kp;
            Ki = ki;
            Kd = kd;

            Invoke((Action)(() =>
            {
                KpText.Text = Kp.ToString();
                KiText.Text = Ki.ToString();
            }
            ));
        }
    }
}

```

```

        KdText.Text = Kd.ToString();
        lblStatus.Text = $"PID Updated → Kp={Kp}, Ki={Ki},
Kd={Kd}";
    }
    }
}
if (parts.Length == 4 && parts[0] == "252")
{
    if (float.TryParse(parts[1], out float kp2) &&
        float.TryParse(parts[2], out float ki2) &&
        float.TryParse(parts[3], out float kd2))
    {
        Kp2 = kp2;
        Ki2 = ki2;
        Kd2 = kd2;

        Invoke((Action)(() =>
        {
            OuterKpText.Text = Kp2.ToString();
            OuterKiText.Text = Ki2.ToString();
            OuterKdText.Text = Kd2.ToString();
            lblStatus.Text = $"PID Updated → Kp2={Kp2}, Ki2={Ki2},
Kd2={Kd2}";
        }
        )));
    }
}
else if (parts.Length == 2 && parts[0] == "253") {
    if (float.TryParse(parts[1], out float Beta)) {
        Invoke((Action)(() =>
        {
            BetaText.Text = Beta.ToString();
            lblStatus.Text = $"Beta Updated → Beta={Beta}";
        }
        )));
    }
}
else if (parts.Length == 2 && parts[0] == "254")
{
    if (float.TryParse(parts[1], out float Offset))
    {
        Invoke((Action)(() =>
        {
            TiltOffsetText.Text = Offset.ToString();
            lblStatus.Text = $"Tilt Offset Updated → Offset={Offset}";
        }
        )));
    }
}
}
}

```

```

// -----
// FORWARD + BACKWARD
// -----
private void speedBar_MouseDown(object sender, MouseEventArgs e)
{
    if (txChar == null)
    {
        lblStatus.Text = "Not connected to BLE";
        return;
    }

    speedTimer.Start();
}
private void speedTimer_Tick(object sender, EventArgs e)
{
    if (txChar == null)
    {
        speedTimer.Stop();
        lblStatus.Text = "Not connected to BLE";
        return;
    }

    string text = $"251 {speedBar.Value} 0";
    SendString(text);
    lblStatus.Text = "Sending text: " + text;
}
private void speedBar_MouseLeave(object sender, EventArgs e)
{
    speedBar.Value = 0;

    if (txChar == null)
    {
        speedTimer.Stop();
        lblStatus.Text = "Not connected to BLE";
        return;
    }

    string text = $"251 0 0";
    SendString(text);
    lblStatus.Text = "Sending text: " + text;
    speedTimer.Stop();
}

// -----
// TURNING
// -----

```

```

private void btnRight_MouseDown(object sender, MouseEventArgs e)
{
    if (txChar == null)
    {
        lblStatus.Text = "Not connected to BLE";
        return;
    }

    string text = $"250 1 1";
    SendString(text);
    lblStatus.Text = "Sending text: " + text;
}

private void btnRight_MouseUp(object sender, MouseEventArgs e)
{
    if (txChar == null)
    {
        lblStatus.Text = "Not connected to BLE";
        return;
    }

    string text = $"250 0 0";
    SendString(text);
    lblStatus.Text = "Sending text: " + text;
}

private void btnLeft_MouseDown(object sender, MouseEventArgs e)
{
    if (txChar == null)
    {
        lblStatus.Text = "Not connected to BLE";
        return;
    }

    string text = $"250 -1 10";
    SendString(text);
    lblStatus.Text = "Sending text: " + text;
}

private void btnLeft_MouseUp(object sender, MouseEventArgs e)
{
    if (txChar == null)
    {
        lblStatus.Text = "Not connected to BLE";
        return;
    }

    string text = $"250 0 0";
    SendString(text);
    lblStatus.Text = "Sending text: " + text;
}

```

```

}

private void btnEstop_Click(object sender, EventArgs e)
{
    if (txChar == null)
    {
        lblStatus.Text = "Not connected to BLE";
        return;
    }

    string text = $"123 234 69";
    SendString(text);
    lblStatus.Text = "Sending text: " + text;
}

private void TuneButton_Click(object sender, EventArgs e)
{
    if (txChar == null)
    {
        lblStatus.Text = "Not connected to BLE";
        return;
    }

    string text = $"255 {KpText.Text} {KiText.Text} {KdText.Text}";
    SendString(text);
    text = $"252 {OuterKpText.Text} {OuterKiText.Text} {OuterKdText.Text}";
    SendString(text);
    lblStatus.Text = "Tune Values Sent";
}

private void SyncButton_MouseDown(object sender, MouseEventArgs e)
{
    if (txChar == null)
    {
        lblStatus.Text = "Not connected to BLE";
        return;
    }

    string text = $"253 1";
    SendString(text);
    lblStatus.Text = "Sync Started";
}

private void SyncButton_MouseUp(object sender, MouseEventArgs e)
{
    if (txChar == null)
    {

```



```

        lblStatus.Text = "Not connected to BLE";
        return;
    }

    string text = "253 " + BetaText.Text;
    SendString(text);
    lblStatus.Text = "Sync Complete";
}

private void AdjustButton_Click(object sender, EventArgs e)
{
    if (txChar == null)
    {
        lblStatus.Text = "Not connected to BLE";
        return;
    }

    string text = "254 " + TiltOffsetText.Text;
    SendString(text);
    //text = "253 " + BetaText.Text;
    //SendString(text);
    lblStatus.Text = "Adjustments Sent";
}

private void speedBar_ValueChanged(object sender, EventArgs e)
{
    if (txChar == null)
    {
        speedTimer.Stop();
        lblStatus.Text = "Not connected to BLE";
        return;
    }

    string text = $"251 {speedBar.Value} 0";
    SendString(text);
    lblStatus.Text = "Sending text: " + text;
}

private void btnFwd_MouseDown(object sender, MouseEventArgs e)
{
    if (txChar == null)
    {
        speedTimer.Stop();
        lblStatus.Text = "Not connected to BLE";
        return;
    }
}

```

```

        string text = $"251 5 0";
        SendString(text);
        lblStatus.Text = "Sending text: " + text;
    }

    private void btnFwd_MouseUp(object sender, MouseEventArgs e)
    {
        if (txChar == null)
        {
            speedTimer.Stop();
            lblStatus.Text = "Not connected to BLE";
            return;
        }

        string text = $"251 0 0";
        SendString(text);
        lblStatus.Text = "Sending text: " + text;
    }

    private void btnFastFwd_MouseDown(object sender, MouseEventArgs e)
    {
        if (txChar == null)
        {
            speedTimer.Stop();
            lblStatus.Text = "Not connected to BLE";
            return;
        }

        string text = $"251 10 0";
        SendString(text);
        lblStatus.Text = "Sending text: " + text;
    }

    private void btnFastFwd_MouseUp(object sender, MouseEventArgs e)
    {
        if (txChar == null)
        {
            speedTimer.Stop();
            lblStatus.Text = "Not connected to BLE";
            return;
        }

        string text = $"251 0 0";
        SendString(text);
        lblStatus.Text = "Sending text: " + text;
    }

```

```

private void btnStop_Click(object sender, EventArgs e)
{
    if (txChar == null)
    {
        speedTimer.Stop();
        lblStatus.Text = "Not connected to BLE";
        return;
    }

    string text = $"251 0 0";
    SendString(text);
    lblStatus.Text = "Sending text: " + text;
}

private void btnBkd_MouseDown(object sender, MouseEventArgs e)
{
    if (txChar == null)
    {
        speedTimer.Stop();
        lblStatus.Text = "Not connected to BLE";
        return;
    }

    string text = $"251 -5 0";
    SendString(text);
    lblStatus.Text = "Sending text: " + text;
}

private void btnBkd_MouseUp(object sender, MouseEventArgs e)
{
    if (txChar == null)
    {
        speedTimer.Stop();
        lblStatus.Text = "Not connected to BLE";
        return;
    }

    string text = $"251 0 0";
    SendString(text);
    lblStatus.Text = "Sending text: " + text;
}

private void btnFastBkd_MouseDown(object sender, MouseEventArgs e)
{
    if (txChar == null)
    {
        speedTimer.Stop();
    }
}

```

```

        lblStatus.Text = "Not connected to BLE";
        return;
    }

    string text = $"251 -10 0";
    SendString(text);
    lblStatus.Text = "Sending text: " + text;
}

private void btnFastBkd_MouseUp(object sender, MouseEventArgs e)
{
    if (txChar == null)
    {
        speedTimer.Stop();
        lblStatus.Text = "Not connected to BLE";
        return;
    }

    string text = $"251 0 0";
    SendString(text);
    lblStatus.Text = "Sending text: " + text;
}
}
}

```

APPENDIX B - ESP32 Firmware

```
#include <SPI.h>
#include <Arduino.h>
#include <FastPID.h>
#include <ESP32Encoder.h>
#include <MadgwickAHRS.h>
#include <BLEDevice.h>
#include <BLEUtils.h>
#include <BLEServer.h>
#include <BLE2902.h>

// Bluetooth
#define SERVICE_UUID          "7bd8e1e7-186b-4442-bd1e-eeedf0bb9139"
#define CHARACTERISTIC_UUID   "0644d551-7b7f-4013-998d-f009450e801b"
#define RX_TILT_UUID          "2bf4bf4d-009c-4447-a34f-f94f22fa2303"

#define STOP1 123
#define STOP2 234
#define STOP3 69

BLECharacteristic *pCharacteristic;
BLECharacteristic *tiltChar;
bool deviceConnected=false;

class MyServerCallbacks : public BLEServerCallbacks
{
    void onConnect(BLEServer* pServer) { deviceConnected = true; }
    void onDisconnect(BLEServer* pServer)
    {
        deviceConnected = false;
        BLEDevice::startAdvertising();
    }
};

// Accelerometer & Gyroscope
#define MPU_ADDR 0x68 // AD0 low
#define SFa 4096 // Scale Factor (Accelerometer)
#define SFg 32.8 // Scale Factor (Gyroscope)
```

```

#define FilterHz 1000.0f // Filter Freq (Hz)
#define PIDHz 500.0f // PID Freq (Hz)
#define PWR_MGMT_1      0x6B // Power management register
#define ACCEL_XOUT_H     0x3B // Start of accel data
#define GYRO_XOUT_H      0x43 // Start of gyro data

// Define SPI pins
#define SPI_CLK  14 // SCLK
#define SPI_MISO 12 // SDO (MISO)
#define SPI_MOSI 13 // SDI (MOSI)
#define SPI_CS   15 // NCS (Chip Select)

// Control Loop
#define max_duty 128
#define max_angle 25
#define max_rpm_sp 5
#define max_current 4.0f // A
#define motor_R 3.0f // Ohms
#define Kv 0.001 // V/rpm

// Encoder Constants
#define Pulse_Rev 374 // Pulse/Rev
#define Enc_A 22
#define Enc_B 21
#define Min_Period 350 // us

// Motor Controller
#define PWM1 32
#define INA1 23
#define INB1 33
#define PWM2 27
#define INA2 19
#define INB2 18

volatile uint32_t IMU_Time = 0;
volatile bool CCW;
volatile unsigned long last_edge_us = 0;
volatile unsigned long edge_period_us = 0;

```

```

float Kp_b = 0.18f, Ki_b = 0.05f, Kd_b = 0.01f;
float Kp_v = 0.1f, Ki_v = 0.8f, Kd_v = 0.0003f;
float vel_sp = 0.0f;
float vel_enc = 0.0f;

float turn_bias = 0.0f;
float bias = 0.83489f; // Reaching setpoint time constant (s)

uint32_t period_copy;
uint32_t last_copy;
bool ccw_copy;

bool eSTOP = false;

Madgwick filter;
hw_timer_t *pidTimer = NULL;
SemaphoreHandle_t pidSem;
QueueHandle_t tiltQueue;
SPISettings mpuSettings(1000000, MSBFIRST, SPI_MODE0);
FastPID balance_pid(Kp_b, Ki_b, Kd_b, PIDHz, 16, true);
FastPID velocity_pid(Kp_v, Ki_v, Kd_v, PIDHz, 16, true);
ESP32Encoder encoder;

// Function to write a byte to a register
void writeRegister(uint8_t reg, uint8_t value) {
    SPI.beginTransaction(mpuSettings);
    digitalWrite(SPI_CS, LOW);
    SPI.transfer(reg); // Register address
    SPI.transfer(value); // Value to write
    digitalWrite(SPI_CS, HIGH);
    SPI.endTransaction();
}

// Function to read a byte from a register
uint8_t readRegister(uint8_t reg) {
    uint8_t value;
    SPI.beginTransaction(mpuSettings);
    digitalWrite(SPI_CS, LOW);
    SPI.transfer(reg | 0x80); // Set MSB to 1 for read

```

```

    value = SPI.transfer(0x00); // Dummy byte to read
    digitalWrite(SPI_CS, HIGH);
    SPI.endTransaction();
    return value;
}

class RxCallbacks : public BLECharacteristicCallbacks
{
    void onWrite(BLECharacteristic *pChar)
    {
        std::string data = pChar->getValue();
        if (data.length() == 0) return;

        // Convert std::string → Arduino String (easier parsing)
        String rx = String(data.c_str());

        // Packet array (change size if needed)
        const int PACKET_SIZE = 5;
        float packet[PACKET_SIZE] = {0};

        int index = 0;
        int start = 0;

        while (index < PACKET_SIZE)
        {
            int spacePos = rx.indexOf(' ', start);

            if (spacePos == -1)
            {
                // Last number
                packet[index++] = rx.substring(start).toFloat();
                break;
            }
            else
            {
                packet[index++] = rx.substring(start, spacePos).toFloat();
                start = spacePos + 1;
            }
        }
    }
}

```



```

//Check eStop case
if(packet[0] == STOP1 && packet[1] == STOP2 && packet[2] == STOP3)
{
    if(eSTOP)
    {
        eSTOP = false;
    }
    else
    {
        digitalWrite(INA1, LOW);
        digitalWrite(INB1, LOW);
        digitalWrite(INA2, LOW);
        digitalWrite(INB2, LOW);
        eSTOP = true;
    }
}
else
{
    if(packet[0] == 255)
    {
        Kp_b = packet[1];
        Ki_b = packet[2];
        Kd_b = packet[3];

        balance_pid.setCoefficients(Kp_b, Ki_b, Kd_b, PIDHz);
    }
    else if(packet[0] == 254)
    {
        bias = packet[1];
    }
    else if(packet[0] == 253)
    {
        filter.beta = packet[1];
    }
    else if(packet[0] == 252)
    {
        Kp_v = packet[1];
        Ki_v = packet[2];
    }
}

```

```

        Kd_v = packet[3];

        velocity_pid.setCoefficients(Kp_v, Ki_v, Kd_v, PIDHz);
        velocity_pid.clear();
    }
    else if(packet[0] == 251)
    {
        vel_sp = packet[1] * 5;
    }
    else if(packet[0] == 250 && vel_sp == 0)
    {
        turn_bias = packet[1] * packet[2];
    }
}
};

void IMUTask(void *pvParameters)
{
    const TickType_t period = pdMS_TO_TICKS(1000 / FilterHz);

    for (;;)
    {
        uint8_t data[14];

        SPI.beginTransaction(mpuSettings);
        digitalWrite(SPI_CS, LOW);
        SPI.transfer(ACCEL_XOUT_H | 0x80);           // start read at 0x3B, read mode
        for (int i = 0; i < 14; i++) {
            data[i] = SPI.transfer(0x00);           // clock out dummy bytes
        }
        digitalWrite(SPI_CS, HIGH);
        SPI.endTransaction();

        // Now unpack (big-endian, signed 16-bit)
        int16_t ax = (int16_t)(data[0] << 8 | data[1]);
        int16_t ay = (int16_t)(data[2] << 8 | data[3]);
        int16_t az = (int16_t)(data[4] << 8 | data[5]);
    }
}

```

```

int16_t gx = (int16_t)(data[8] << 8 | data[9]);
int16_t gy = (int16_t)(data[10] << 8 | data[11]);
int16_t gz = (int16_t)(data[12] << 8 | data[13]);

//ADC Counts to Units (g)
float ax_ = (float)ax / SFa - 0.04f;
float ay_ = (float)ay / SFa;
float az_ = (float)az / SFa + 0.02f;

//ADC Counts to Units (deg/s)
float gx_ = (float)gx / SFg + 3.3f;
float gy_ = (float)gy / SFg + 1.85f;
float gz_ = (float)gz / SFg - 0.3f;

//Madgwick Filter
filter.invSampleFreq = (micros() - IMU_Time) * 1e-6f;
filter.updateIMU(gx_, gy_, gz_, ax_, ay_, az_);

IMU_Time = micros();

float tilt_est = filter.getPitch();
xQueueOverwrite(tiltQueue, &tilt_est);

vTaskDelay(period);
}
}

void IRAM_ATTR onPidTimer()
{
    xSemaphoreGiveFromISR(pidSem, NULL);
}

void pidTask(void *pvParameters)
{
    uint32_t last_us = micros();

    for (;;)
    {
        xSemaphoreTake(pidSem, portMAX_DELAY);
    }
}

```

```

uint32_t now = micros();
float SampleT = (now - last_us) * 1e-6f;

last_us = now;

static long previousCount = 0;
long currentCount = encoder.getCount();
long deltaCount = currentCount - previousCount;
previousCount = currentCount;

// For velocity calculation
static float vel_enc = 0.0;

vel_enc = 60.0 * (float)deltaCount / (4.0 * Pulse_Rev) / SampleT;

// Double complementary filter
static float vel_filt1 = 0.0f, vel_filt2 = 0.0f;
const float a_vel = 0.2f;
vel_filt1 = a_vel * vel_enc + (1.0 - a_vel) * vel_filt1;
vel_filt2 = a_vel * vel_filt1 + (1.0 - a_vel) * vel_filt2;

float vel_smooth = vel_filt2;

// Scale inputs for better resolution (FastPID uses int16_t)
int16_t vel_sp_scaled = (int16_t)(vel_sp * 75.0f); // x75 RPM
int16_t vel_meas_scaled = (int16_t)(vel_smooth * 75.0f);

// Outer loop: Velocity PID → desired tilt setpoint
int16_t tilt_sp_scaled = velocity_pid.step(vel_sp_scaled, vel_meas_scaled);

// Calculate angle setpoint
float output_sp = (float)tilt_sp_scaled / 75.0f + bias;

float local_tilt = 0.0f;
xQueuePeek(tiltQueue, &local_tilt, 0);

// Scale tilt for inner PID

```

```

    int16_t tilt_sp_scaled_inner = (int16_t)(output_sp * 100.0f); // x100
degrees
    int16_t tilt_meas_scaled      = (int16_t)(local_tilt * 100.0f);

    // Inner loop: Angle PID → motor input current
    int16_t current_scaled = balance_pid.step(tilt_sp_scaled_inner,
tilt_meas_scaled);
    float current_des = (int)current_scaled / 100.0f;

    // Calculate scaled duty cycle based on motor armature circuit
    float output_duty = (current_des * motor_R - Kv * vel_smooth) * 255.0f /
12.0f;
    output_duty = constrain(output_duty, -max_duty, max_duty);

    //Motor Control
    if(!eSTOP)
    {
        static bool fwd = true;

        //Set Direction
        if(output_duty < 0 && fwd)
        {
            digitalWrite(INA1, LOW);
            digitalWrite(INB1, HIGH);
            digitalWrite(INA2, HIGH);
            digitalWrite(INB2, LOW);

            fwd = false;
        }
        else if(output_duty > 0 && !fwd)
        {
            digitalWrite(INA1, HIGH);
            digitalWrite(INB1, LOW);
            digitalWrite(INA2, LOW);
            digitalWrite(INB2, HIGH);

            fwd = true;
        }
    }

```

```

        //Write Velocity
        if(abs(local_tilt) > max_angle)
        {
            digitalWrite(INA1, HIGH);
            digitalWrite(INB1, HIGH);
            digitalWrite(INA2, HIGH);
            digitalWrite(INB2, HIGH);
        }
        else
        {
            analogWrite(PWM1, abs(output_duty));
            analogWrite(PWM2, abs(output_duty + turn_bias));
        }
    }
}

// ----- Setup -----
void setup() {
    Serial.begin(115200);
    filter.begin(FilterHz);

    // Initialize SPI
    SPI.begin(SPI_CLK, SPI_MISO, SPI_MOSI, SPI_CS);
    pinMode(SPI_CS, OUTPUT);
    digitalWrite(SPI_CS, HIGH); // Deselect chip initially

    Serial.printf("Reset reason: %d\n", (int)esp_reset_reason());

    delay(100);

    // Wake up the IMU (clear SLEEP bit)
    writeRegister(PWR_MGMT_1, 0x00);

    // Set Bandwidth
    writeRegister(0x1A, 0x02); // DLPF_CFG = 2 for ~98Hz bandwidth (Gyroscope)
    writeRegister(0x1D, 0x02); // DLPF_CFG = 2 for ~98Hz bandwidth (Accelerometer)

    // Set full-scale ranges and bandwidth

```

```

writeRegister(0x1B, 0x10); // Gyro ±1000 dps
writeRegister(0x1C, 0x10); // Accel ±8g

//Bluetooth Setup
BLEDevice::init("Grass");

BLEServer *pServer = BLEDevice::createServer();
pServer->setCallbacks(new MyServerCallbacks());

BLEService *pService = pServer->createService(SERVICE_UUID);

pCharacteristic = pService->createCharacteristic(
    CHARACTERISTIC_UUID,
    BLECharacteristic::PROPERTY_WRITE |
    BLECharacteristic::PROPERTY_READ
);

pCharacteristic->setCallbacks(new RxCallbacks());
pCharacteristic->setValue("");

// Tilt notify characteristic
tiltChar = pService->createCharacteristic(
    RX_TILT_UUID,
    BLECharacteristic::PROPERTY_NOTIFY
);
tiltChar->addDescriptor(new BLE2902());

pService->start();

BLEAdvertising *pAdvertising = BLEDevice::getAdvertising();
pAdvertising->addServiceUUID(SERVICE_UUID);
BLEDevice::startAdvertising();

//PID Setup
balance_pid.setOutputRange(-max_current * 100.0f, max_current * 100.0f);
balance_pid.setCoefficients(Kp_b, Ki_b, Kd_b, PIDHz);

velocity_pid.setOutputRange(-max_angle * 100.0f, max_angle * 100.0f);
velocity_pid.setCoefficients(Kp_v, Ki_v, Kd_v, PIDHz);

```

```

//Motor Controller Setup
pinMode(PWM1, OUTPUT);
pinMode(INA1, OUTPUT);
pinMode(INB1, OUTPUT);
pinMode(PWM2, OUTPUT);
pinMode(INA2, OUTPUT);
pinMode(INB2, OUTPUT);

analogWrite(PWM1, 0);
analogWrite(PWM2, 0);

pidSem = xSemaphoreCreateBinary();
tiltQueue = xQueueCreate(1, sizeof(float));

// Enc setup
encoder.attachFullQuad(Enc_A, Enc_B);
encoder.setFilter(1000);
encoder.clearCount();

// IMU task to Core 0
xTaskCreatePinnedToCore(IMUTask, "IMU", 4096, NULL, 5, NULL, 0);

// PID task to Core 1
xTaskCreatePinnedToCore(pidTask, "PID", 4096, NULL, 10, NULL, 1);

// Hardware timer (500 Hz)
pidTimer = timerBegin(0, 80, true); // 1  $\mu$ s tick
timerAttachInterrupt(pidTimer, &onPidTimer, true);
timerAlarmWrite(pidTimer, 2000, true); // 1.333 ms
timerAlarmEnable(pidTimer);
}

//Main Loop
void loop()
{
    vTaskDelay(pdMS_TO_TICKS(100));
}

```


APPENDIX C - Key Resources

Inverted Pendulum Physics Derivation:

<https://karooza.net/modelling-an-inverted-pendulum>